

Experiment 1: 4 Bit adder

Aim: To write a verilog code for 4bit adder and verify the functionality using Test bench.

- Synthesize, Analyse Reports and Netlist, Critical Path and Max Operating Frequency.
- From the report generated find the total number of cells, power requirement and total area requirement.

Tool Required:

- Functional Simulation: Incisive Simulator (ncvlog, ncelab, ncsim)
- Synthesis: Genus

Design Information and Block Diagram:

A full adder is a combinational circuit that performs the arithmetic sum of three input bits A_i , addend B_i and carry in C in from the previous adder. Its results contain the sum S_i and the carry out, C out to the next stage. So to design a 4-bit adder circuit we start by designing the 1-bit full adder then connecting the four 1-bit full adders to get the 4-bit adder as shown in the diagram below. For the 1-bit full adder, the design begins by drawing the Truth Table for the three input and the corresponding output

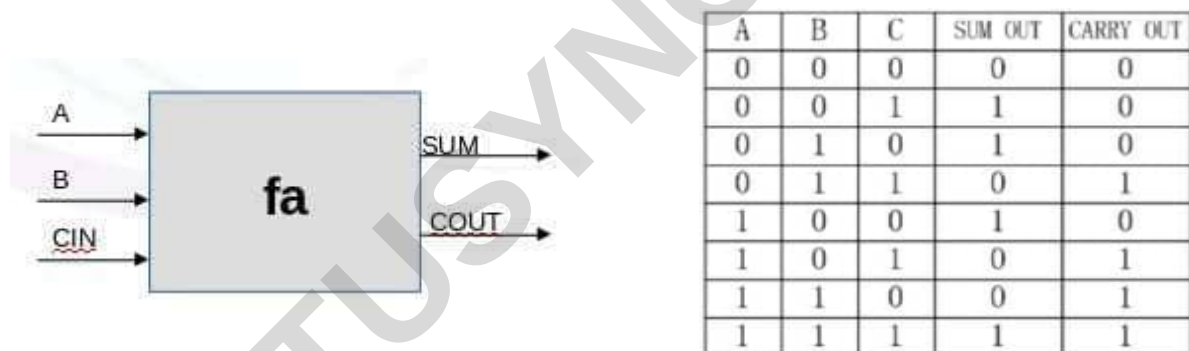


Fig: Diagram and truth table of full adder

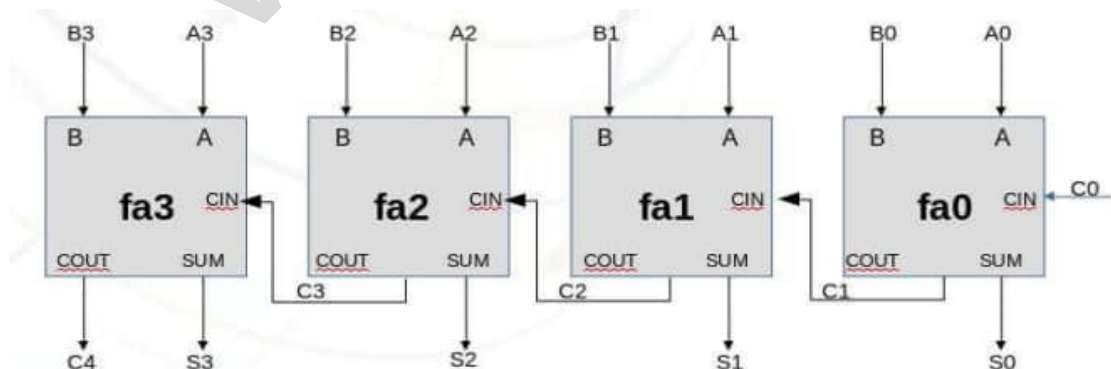


Fig: Diagram of 4 Bit Adder

Verilog code:**Source Code – fa.v:**

```
module full_adder( A, B, CIN, S, COUT );  
    input A, B, CIN;  
    output S, COUT;  
    assign S = A ^ B ^ CIN;    // Sum calculation  
    assign COUT = (A & B) | (CIN & (A ^ B)); // Carry-out calculation  
endmodule
```

Source Code – fa_4bit.v:

```
module four_bit_adder(A, B, C0, S, C4);  
    input [3:0] A, B; // Inputs A and B are 4-bit each  
    input C0;        // Carry-in (single bit)  
    output [3:0] S;  // 4-bit sum output  
    output C4;       // Carry-out (single bit)  
  
    wire C1, C2, C3; // Intermediate carry wires  
  
    // Instantiating full adders for each bit position  
    full_adder fa0 (A[0], B[0], C0, S[0], C1);  
    full_adder fa1 (A[1], B[1], C1, S[1], C2);  
    full_adder fa2 (A[2], B[2], C2, S[2], C3);  
    full_adder fa3 (A[3], B[3], C3, S[3], C4);  
  
endmodule
```

Testbench Code:**Test Bench – fa_test.v:**

```
module test_4_bit;
    reg [3:0] A, B; // 4-bit inputs
    reg C0;        // Single-bit carry-in
    wire [3:0] S;  // 4-bit sum output
    wire C4;       // Single-bit carry-out

    // Instantiate the 4-bit adder module
    four_bit_adder dut (A, B, C0, S, C4);

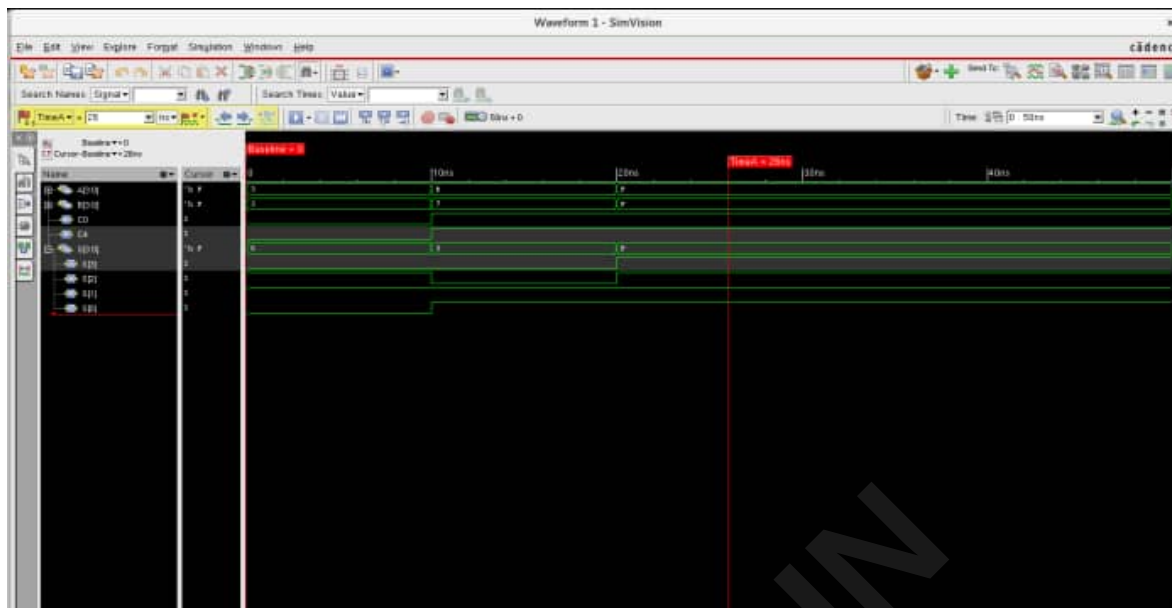
    // Test stimulus
    initial begin
        $monitor("Time=%0t | A=%b B=%b C0=%b | S=%b C4=%b", $time, A,
        B, C0, S, C4);

        A = 4'b0011; B = 4'b0011; C0 = 1'b0; #10; // 3 + 3 + 0 = 6 (0110)
        A = 4'b1011; B = 4'b0111; C0 = 1'b1; #10; // 11 + 7 + 1 = 19 (10011)
        A = 4'b1111; B = 4'b1111; C0 = 1'b1; #10; // 15 + 15 + 1 = 31 (11111)

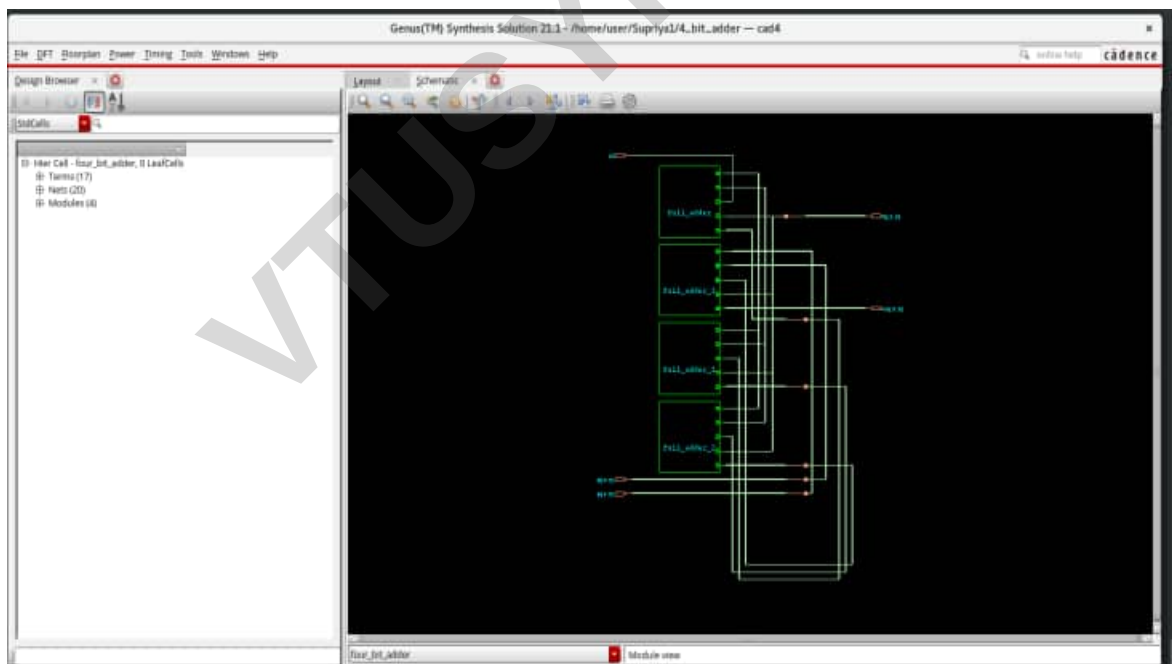
        #20 $finish; // End simulation
    end
endmodule
```

Output:

Waveform:



RTL Schematic:



Experiment 2: 4bit Shift and Add Multiplier

Aim: To write a verilog code for 4bit shift and add multiplier and verify the functionality using Test bench.

- Synthesize and analyze reports and netlist, as well as critical path and maximum delay.
- From the report generated, find the total number of cells, power requirement and total area requirement.

Tool Required:

- Functional Simulation: Incisive Simulator (ncvlog, ncelab, ncsim)
- Synthesis: Genus

Design Information and Block Diagram:

A 4-bit shift and add multiplier is a digital circuit designed to multiply two 4-bit binary numbers by sequentially shifting and adding, mimicking the long multiplication method used in arithmetic. It consists of a multiplicand, which is the number being multiplied, and a multiplier, which dictates how many times the multiplicand is added. The process begins by storing both numbers in separate registers and initializing a product register to zero. The multiplication proceeds bit by bit, starting from the least significant bit (LSB) of the multiplier. If the current bit is 1, the multiplicand is added to the product register; if the bit is 0, no addition is performed. After each step, the multiplicand is left-shifted to align with the next bit of the multiplier, and the multiplier is right-shifted to bring the next bit into position. This process continues for four cycles, as each bit of the multiplier is processed sequentially.

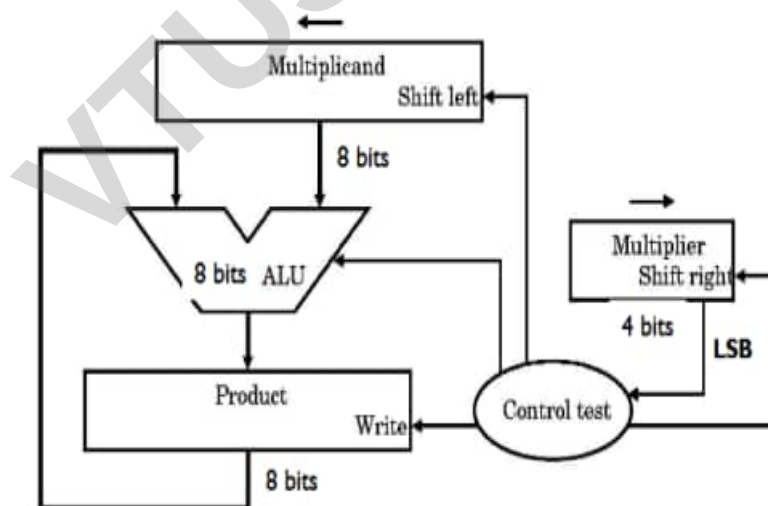


Fig.1 Block diagram

Calculation :

$$\begin{array}{r} 1011 \text{ (Multiplicand)} \\ \times 1101 \text{ (Multiplier)} \\ \hline 1011 \quad (1011 \times 1) \\ + 00000 \quad (1011 \times 0, \text{ shift left}) \\ ++101100 \quad (1011 \times 1, \text{ shift left twice}) \\ +++1011000 \quad (1011 \times 1, \text{ shift left thrice}) \\ \hline 10001111 \quad (\text{Product} = 143 \text{ in decimal}) \end{array}$$

Verilog Code

```
module shift_add_multiplier (  
    input [3:0] a, // 4-bit multiplicand  
    input [3:0] b, // 4-bit multiplier  
    output [7:0] product  
);  
  
    // Internal registers  
    reg [7:0] prod_reg;  
    reg [3:0] count;  
  
    // On each clock cycle, we check one bit of b and add/shift  
    // This example uses a simple sequential approach with a for-loop.  
  
    integer i;  
    always @* begin  
        // Initialize the product register  
        prod_reg = 8'b0;  
  
        // Loop through each bit of b  
        for (i = 0; i < 4; i = i + 1) begin  
            // If the current bit of b is 1, add the partial product  
            if (b[i] == 1'b1) begin  
                // Shift a by i bits to the left and add  
                prod_reg = prod_reg + (a << i);  
            end  
        end  
        assign product = prod_reg;  
    end  
endmodule
```

Testbench Code:

```

`timescale 1ns/1ps // Define time unit and precision

module tb_shift_add_multiplier;

    // Testbench signals
    reg [3:0] a_tb;
    reg [3:0] b_tb;
    wire [7:0] product_tb;

    // Instantiate the design under test (DUT)
    shift_add_multiplier dut (
        .a(a_tb),
        .b(b_tb),
        .product(product_tb)
    );

    initial begin
        // Display header
        $display("Time\t a\t b\t | product (Binary) | Product (Decimal)");
        $display("-----");

        // Test cases
        a_tb = 4'b0000; b_tb = 4'b0000; #10;
        $display("%0dns:\t %b\t %b\t | %b\t | %0d", $time, a_tb, b_tb, product_tb, product_tb);

        a_tb = 4'b0001; b_tb = 4'b0001; #10;
        $display("%0dns:\t %b\t %b\t | %b\t | %0d", $time, a_tb, b_tb, product_tb, product_tb);

        a_tb = 4'b0010; b_tb = 4'b0011; #10;
        $display("%0dns:\t %b\t %b\t | %b\t | %0d", $time, a_tb, b_tb, product_tb, product_tb);

        a_tb = 4'b1010; b_tb = 4'b0011; #10;
        $display("%0dns:\t %b\t %b\t | %b\t | %0d", $time, a_tb, b_tb, product_tb, product_tb);

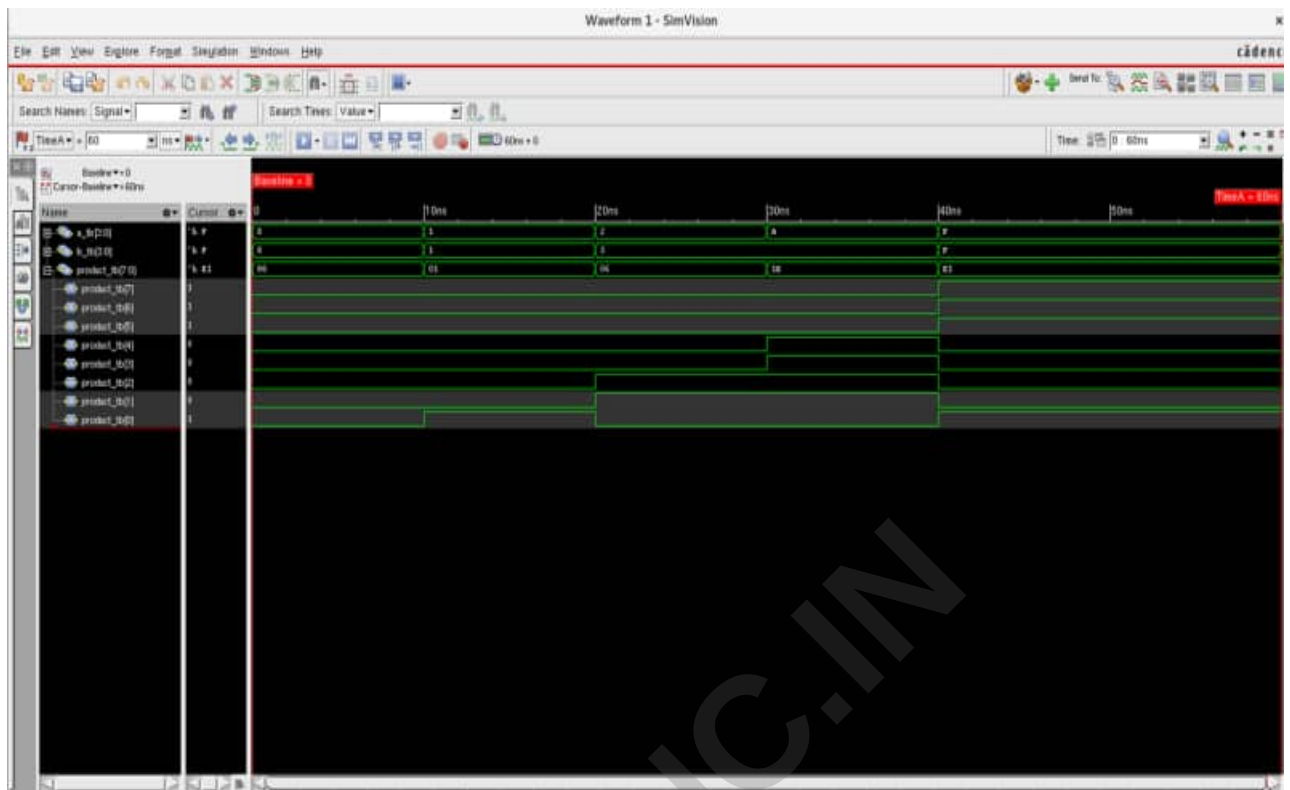
        a_tb = 4'b1111; b_tb = 4'b1111; #10;
        $display("%0dns:\t %b\t %b\t | %b\t | %0d", $time, a_tb, b_tb, product_tb, product_tb);

        // End simulation
        #10;
        $stop;
    end

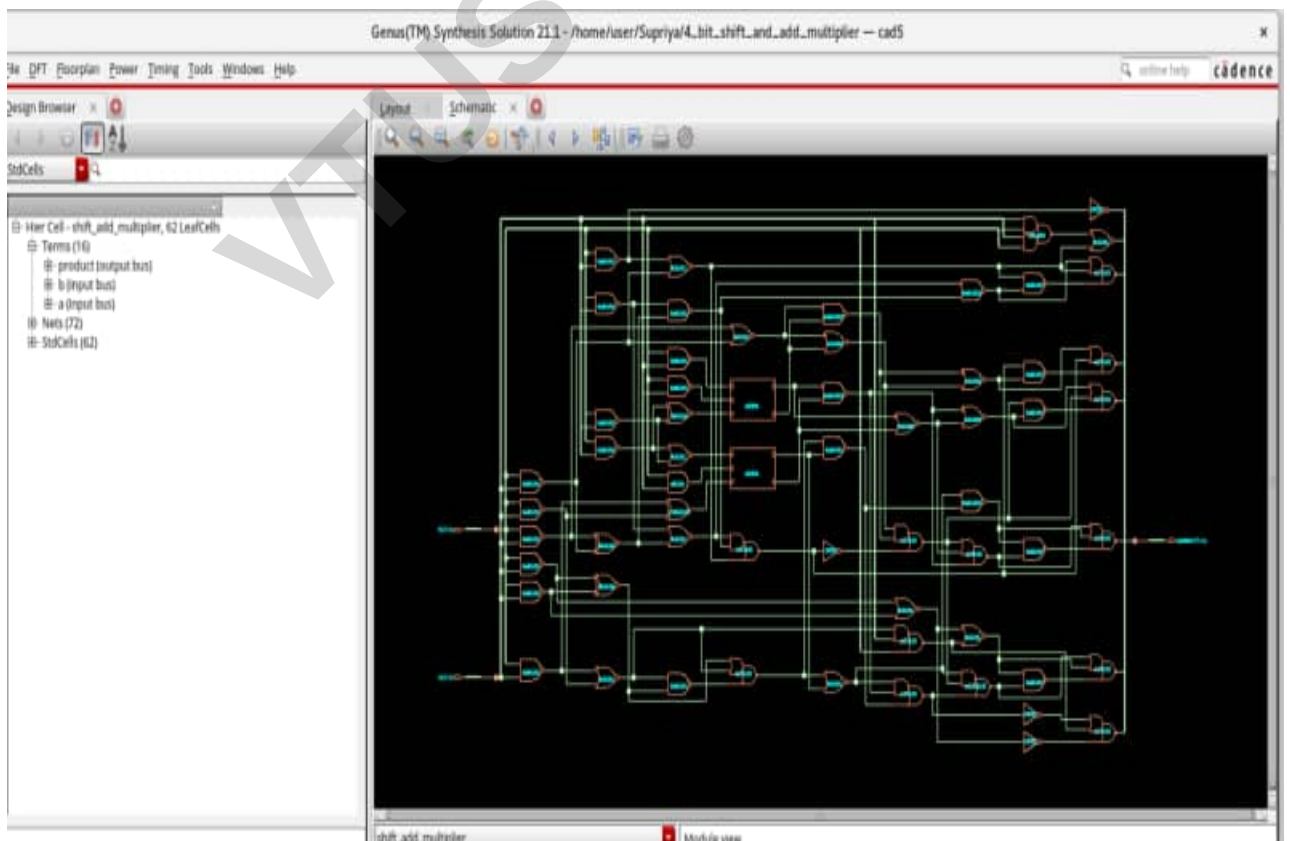
endmodule

```

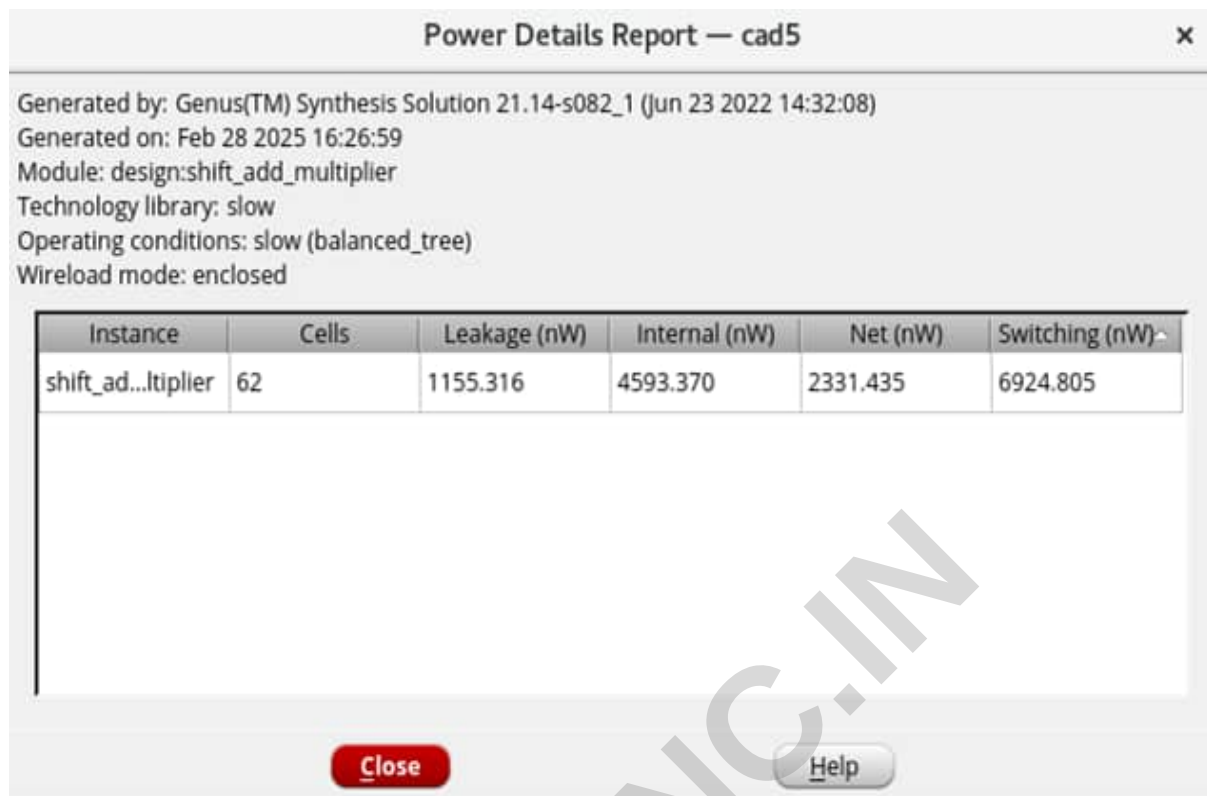

**Output:
Waveform:**



RTL Schematic:



Power Requirements (power details report):



Timing Report:

```
@genus:root: 21> report_timing -unconstrained
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Feb 28 2025 04:31:50 pm
Module:            shift_add_multiplier
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Path 1: UNCONSTRAINED
  Startpoint: (R) b[2]
    Endpoint: (R) product[6]

      Capture      Launch
  Drv Adjust: +    0      0

  Data Path: -    1275

#-----#
# Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival Instance
#              (ff) (ps) (ps) (ps) (ps) Location
#-----#
b[2]           -      -   R   (arrival)  4  6.4    0    0    0  (-,-)
g1119_7482/Y   -      A->Y F   NAND2XL  2  5.2   143  109  109  (-,-)
g1102_2802/Y   -      B->Y F   XNOR2X1  1  4.9    75  178  287  (-,-)
g1087_1666/S   -      CI->S R   ADDFX1  2  3.4    87  391  678  (-,-)
g1084_5115/Y   -      B->Y F   NAND2BXL 2  3.3   106  98  776  (-,-)
g1075_5122/Y   -      A1->Y R   AOI21XL  4  6.7   234  205  981  (-,-)
g1066_6783/Y   -      A1N->Y R   AOI2BB1XL 2  3.3   130  173  1153  (-,-)
g1063_6260/Y   -      B->Y F   NAND2XL  1  1.7    84  81  1234  (-,-)
g1061_2398/Y   -      B0->Y R   AOI21XL  1  0.0    57  41  1275  (-,-)
product[6]     -      -   R   (port)    -  -     -    0  1275  (-,-)
#-----#
```

Total Area required:

```

user@cad54.bit_shift_and_add_multiplier
File Edit View Search Terminal Help
@genus:root: 22> report_area
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s002_1
Generated on:      Feb 28 2025  04:34:53 pm
Module:            shift_add_multiplier
Operating conditions: slow (balanced tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

  Instance      Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
shift_add_multiplier      62    277.025    0.000    277.025 <none> (b)
(D) = wireload is default in technology library

```

Total Power Requirement:

Leakage Power: 1155.316 nW
 Internal Power: 4593.370 nW
 Net Power: 2331.435 nW
 Switching Power: 6924.805 nW

Total Area Requirement:

Cell Area: 277.025
 Net Area: 0.000
 Total Area: 277.025

Maximum Delay: 1275ps

Critical path Delay: 1275ps

Critical path: b[2]- NAND2XL- XOR2X1- ADDFX1-NAND2BXL-AOI21XL-AOI2BB1XL-NAND2XL-OAI21XL-product[6]

Experiment 3: 32 bit ALU

Aim: 32-Bit ALU Supporting 4-Logical and 4-Arithmetic operations, using case and if statement for

ALU Behavioral Modeling

- Write Verilog description
- Verify functionality using Test-bench
- Synthesize the design targeting suitable library and by setting area and timing constraints
- Tabulate the Area, Power and Delay for the Synthesized netlist
- Identify Critical path

Tool Required:

- Functional Simulation: Incisive Simulator (ncvlog, ncelab, ncsim)
- Synthesis: Genus

Design Information and Block Diagram:

An Arithmetic Logic Unit (ALU) is a fundamental component of a processor that performs both arithmetic and logical operations. It is a combinational circuit that takes input operands, processes them based on a control signal, and produces an output. In this experiment, we design a 32-bit ALU that supports four logical operations (AND, OR, XOR, NOR) and four arithmetic operations (Addition, Subtraction, Multiplication, and Division) using a behavioral modeling approach in Verilog. The selection of the required operation is controlled by a 3-bit ALU_Sel signal, allowing the ALU to perform different computations based on its value.

The ALU consists of several key components, including a multiplexer (MUX) to select the desired operation, an arithmetic circuit to handle addition, subtraction, multiplication, and division, and a logical circuit to process AND, OR, XOR, and NOR operations. Additionally, the ALU implements a Zero Flag, which is set when the output result is zero. This flag is useful for decision-making processes in microprocessors, where conditional branching may depend on the result of an ALU operation.

The Verilog design is implemented using behavioral modeling, which allows the use of case and if statements for defining the ALU's operations. This approach makes the design flexible, readable, and easy to modify. Behavioral modeling abstracts the internal logic and focuses on functionality, making it an efficient choice for ALU implementation. The ALU operations are encoded as follows: 000 for AND, 001 for OR, 010 for XOR, 011 for NOR, 100 for addition, 101 for subtraction, 110 for multiplication, and 111 for division. The division operation is

carefully implemented to handle cases where the denominator is zero, ensuring error-free computation.

To verify the correctness of the ALU, a testbench is written in Verilog, where multiple test cases are executed to validate the functionality of each operation. Once verified, the ALU is synthesized by targeting an appropriate library while setting area and timing constraints. The synthesized netlist is analyzed to extract key performance metrics, including area, power consumption, and delay. Furthermore, the critical path is identified to determine the slowest combinational logic path, which impacts the ALU's overall performance.

This experiment provides insights into digital design, combinational circuits, and hardware description languages (HDL). The implementation of a 32-bit ALU is essential in understanding the working principles of microprocessors and their arithmetic logic capabilities. By synthesizing the design and analyzing key metrics, we gain practical knowledge about optimizing hardware for efficiency and speed, which is crucial in modern computing systems.

Calculation:

1. Logical Operations

AND (f = 000): Performs bitwise AND.

Example: A = 15 (0000...1111), B = 3 (0000...0011)

Calculation: A & B = 3 (0000...0011)

OR (f = 001): Performs bitwise OR.

Calculation: A | B = 15 (0000...1111)

NOR (f = 010): Performs bitwise NOR (NOT OR).

Calculation: $\sim(A | B) = -16$ (1111...0000)

NAND (f = 011): Performs bitwise NAND (NOT AND).

Calculation: $\sim(A \& B) = -4$ (1111...1100)

2. Arithmetic Operations

Addition (f = 100): Adds A and B.

Calculation: A + B = 15 + 3 = 18

Subtraction (f = 101): Subtracts B from A.

Calculation: A - B = 15 - 3 = 12

Multiplication (f = 110): Multiplies A and B.

Calculation: A * B = 15 × 3 = 45

Division (f = 111): Divides A by B, handling division by zero.

Calculation: A / B = 15 ÷ 3 = 5 (if B ≠ 0, otherwise 0)

Verilog code

```

module alu_32bit_case(y,a,b,f);
input [31:0]a;
input [31:0]b;
input [2:0]f;
output reg [31:0]y;
always@(*)
begin
  case(f)
    3'b000:y=a&b;
    3'b001:y=a|b;
    3'b010:y=~(a|b);
    3'b011:y=~(a&b);
    3'b100:y=a+b;
    3'b101:y=a-b;
    3'b110:y=a*b;
    default:y=32'bx;
  endcase
end
endmodule

```

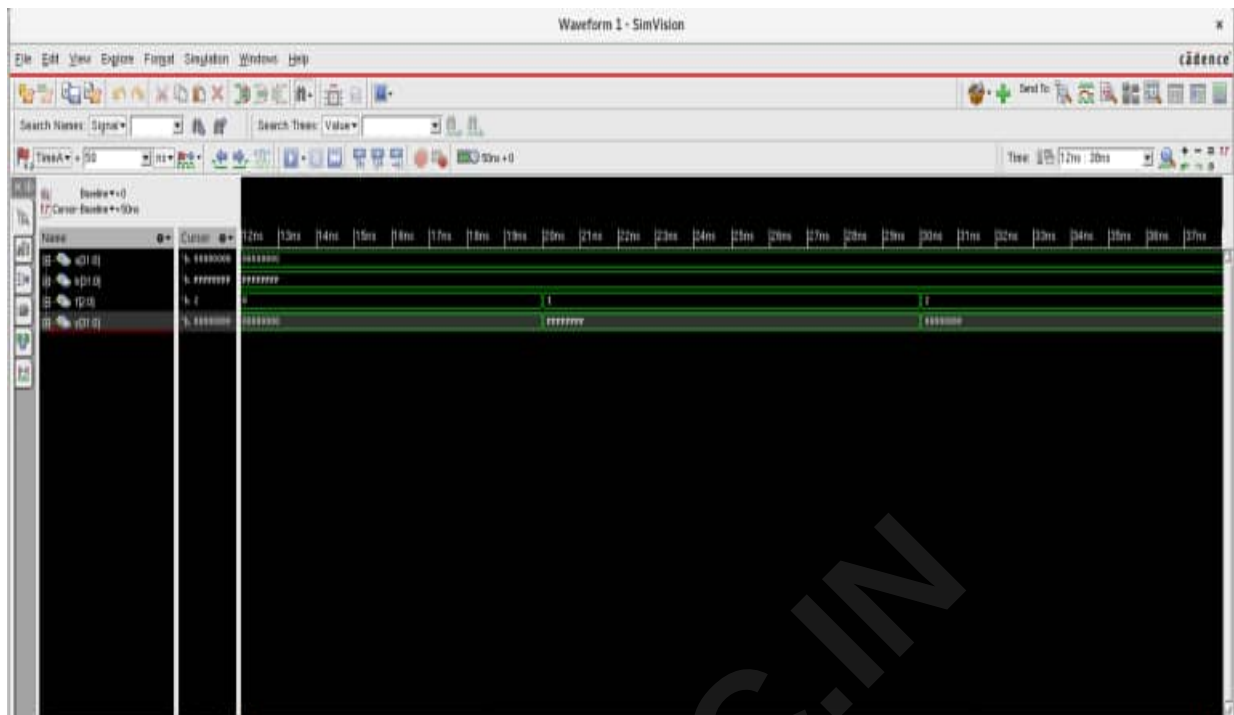
Testbench Code:

```

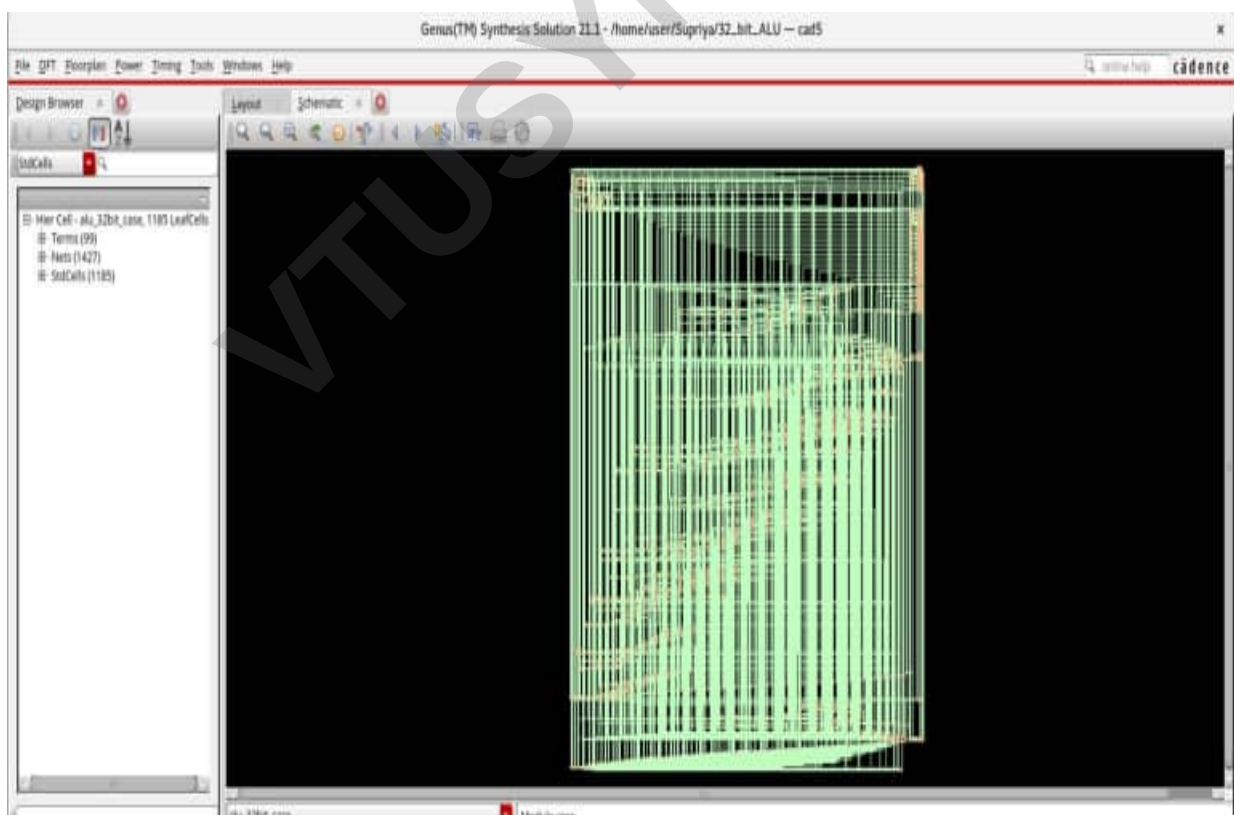
module alu_32bit_tb_case;
reg [31:0]a;
reg [31:0]b;
reg [2:0]f;
wire [31:0]y;
alu_32bit_case test2(.y(y), .a(a), .b(b), .f(f));
initial
begin
  a=32'h00000000;
  b=32'hFFFFFFFF;
  #10 f=3'b000;
  #10 f=3'b001;
  #10 f=3'b010;
  #10 f=3'b010;
  #10 f=3'b011;
  #10 f=3'b100;
end
initial
#50 $finish;
endmodule

```

Output Waveform:



RTL Schematic:



Power Requirements (power details report)

Power Details Report — cad5					
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1 (Jun 23 2022 14:32:08)					
Generated on: Mar 10 2025 17:09:47					
Module: design:alu_32bit_case					
Technology libraries: slow					
Operating conditions: slow (balanced_tree)					
Wireload mode: enclosed					
Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
alu_32bit_case	1185	31248.330	480525.160	209058.024	689583.184

Timing Report:

user@cad5:32_bit_ALU											
File Edit View Search Terminal Help											
#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance Location
#											
	b[2]	-	-	F	(arrival)	40	68.7	0	0	0	(-, -)
	mul_15_13_g7645/Y	-	A->Y	R	CLKINVX1	38	65.3	542	378	378	(-, -)
	mul_15_13_g7544/Y	-	S0->Y	R	MXI2XL	2	3.3	211	216	593	(-, -)
	mul_15_13_g7508_7482/Y	-	A->Y	F	NAND2XL	28	43.4	1028	748	1342	(-, -)
	mul_15_13_g7493/Y	-	A->Y	R	CLKINVX1	2	3.2	227	155	1496	(-, -)
	mul_15_13_g7401_1881/Y	-	A0->Y	F	OAI21XL	1	3.7	237	146	1642	(-, -)
	mul_15_13_g7283_1705/Y	-	S0->Y	F	MXI2XL	2	5.3	241	229	1871	(-, -)
	mul_15_13_g7242_6417/Y	-	S0->Y	F	MXI2XL	1	6.7	267	251	2122	(-, -)
	mul_15_13_cdnfadd3_2346/C0	-	A->C0	F	ADDFX1	1	4.9	97	300	2422	(-, -)
	mul_15_13_cdnfadd4_1666/C0	-	CI->C0	F	ADDFX1	1	4.9	97	250	2672	(-, -)
	mul_15_13_cdnfadd5_7410/C0	-	CI->C0	F	ADDFX1	1	4.9	97	250	2922	(-, -)
	mul_15_13_cdnfadd6_6417/C0	-	CI->C0	F	ADDFX1	1	4.9	97	250	3173	(-, -)
	mul_15_13_cdnfadd7_5477/C0	-	CI->C0	F	ADDFX1	1	4.9	97	250	3423	(-, -)
	mul_15_13_cdnfadd8_2398/C0	-	CI->C0	F	ADDFX1	1	4.9	97	250	3673	(-, -)
	mul_15_13_cdnfadd9_5107/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	3923	(-, -)
	mul_15_13_cdnfadd10_6260/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	4173	(-, -)
	mul_15_13_cdnfadd11_4319/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	4423	(-, -)
	mul_15_13_cdnfadd12_8428/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	4673	(-, -)
	mul_15_13_cdnfadd13_5526/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	4923	(-, -)
	mul_15_13_cdnfadd14_6783/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	5173	(-, -)
	mul_15_13_cdnfadd15_3680/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	5423	(-, -)
	mul_15_13_cdnfadd16_1617/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	5672	(-, -)
	mul_15_13_cdnfadd17_2802/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	5922	(-, -)
	mul_15_13_cdnfadd18_1705/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	6172	(-, -)
	mul_15_13_cdnfadd19_5122/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	6422	(-, -)
	mul_15_13_cdnfadd20_8246/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	6672	(-, -)
	mul_15_13_cdnfadd21_7098/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	6922	(-, -)
	mul_15_13_cdnfadd22_6131/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	7172	(-, -)
	mul_15_13_cdnfadd23_1881/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	7422	(-, -)
	mul_15_13_cdnfadd24_5115/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	7672	(-, -)
	mul_15_13_cdnfadd25_7482/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	7922	(-, -)
	mul_15_13_cdnfadd26_7482/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	7922	(-, -)
	mul_15_13_cdnfadd27_6161/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	8421	(-, -)
	mul_15_13_cdnfadd28_9315/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	8671	(-, -)
	mul_15_13_cdnfadd29_9945/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	8921	(-, -)
	mul_15_13_cdnfadd30_2883/C0	-	CI->C0	F	ADDFX1	1	4.9	96	250	9171	(-, -)
	mul_15_13_cdnfadd31_2346/S	-	CI->S	R	ADDFX1	1	1.7	71	382	9554	(-, -)
	g3847_4733/Y	-	B->Y	F	NAND2XL	1	1.7	69	70	9623	(-, -)
	g3773_1881/Y	-	C0->Y	R	OAI211XL	1	0.0	183	43	9666	(-, -)
	y[31]	-	-	R	(port)	-	-	-	0	9666	(-, -)

Area Report:

```
#-----
@genus:root: 8> report_area
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Mar 12 2025  03:48:43 pm
Module:           alu_32bit_case
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

Instance  Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
alu_32bit_case  1185  8948.829   0.000   8948.829 <none> (D)
```

Power Requirements

- **Leakage Power:** 31,248.330 nW
- **Internal Power:** 480,525.160 nW
- **Net Power:** 209,058.024 nW
- **Switching Power:** 689,583.184 nW

Area Requirements

- **Cell Count:** 1,185
- **Cell Area:** 8,948.829
- **Net Area:** 0.000
- **Total Area:** 8,948.829

Timing and Critical Path

- **Maximum Delay:** 9,666 ps
- **Critical Path Delay:** 9,666 ps
- **Critical Path:**
 - Starts from b[2]
 - Passes through NAND2XL, XOR2X1, ADDFX1, NAND2BXL, AOI21XL, AOI2BB1XL, NAND2XL, OAI21XL
 - Ends at y[31]

Experiment 4: Flipflops

What are flip flops in electronics? A flip-flop is an electronic circuit that can store single-bit binary data either logic 0 or logic 1. Basically, a flip flop is a Bistable multivibrator that changes its output depending on the input. Flip Flops are of two types edge triggered, and level triggered. State of an Edge triggered flip flop changes during the positive or negative edge of a clock cycle. Whereas in the case of level-triggered flip flop output can change during high or low clock duration. Level triggered flip flops are called latch, they are transparent because when they are enabled their output becomes the same as the input.

D-Flipflop:

Aim: For D- Flipflop.

- Write Verilog description
- Verify functionality using Test-bench
- Synthesize the design targeting suitable library and by setting area and timing constraints
- Tabulate the Area, Power and Delay for the Synthesized netlist
- Identify Critical path

Theory:

A D Flip-Flop (Data or Delay Flip-Flop) is a sequential circuit used for storing and transferring data in digital systems. It operates based on a clock signal, capturing the value of the D (data) input at the moment of a clock transition and holding this value until the next clock edge. The Q (output) follows the D input on the rising or falling edge of the clock, ensuring reliable data storage. An optional reset (RST) input can be used to force the output Q to 0, regardless of the clock state. The truth table of the D Flip-Flop shows that when the clock edge occurs, the output Q takes the value of D, while its complement Q' takes the opposite value. This behavior makes the D Flip-Flop a fundamental building block in memory units, shift registers, counters, and finite state machines. It can be implemented using logic gates, as a modification of the SR Flip-Flop, or programmed using Verilog/VHDL for FPGA and ASIC designs. D Flip-Flops are commonly used in clock synchronization, data storage, and frequency division applications. Their ability to store and transfer data reliably makes them essential in modern digital electronics.

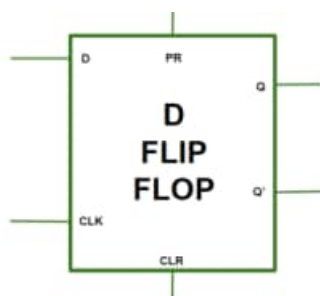


Fig.1 Block Diagram of D-FF

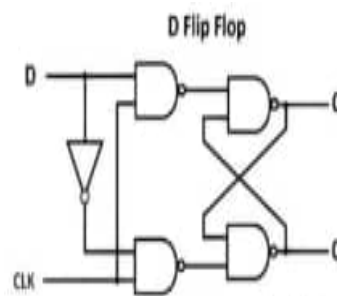


Fig. 1.1 D-FF using NAND gate.

Truth Table of D- Flipflop

Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

Verilog Code:

```

module RisingEdge_DFlipFlop_SyncReset(D,clk,sync_reset,Q);
input D; // Data input
input clk; // clock input
input sync_reset; // synchronous reset
output reg Q; // output Q
always @(posedge clk)
begin
if(sync_reset==1'b1)
Q <= 1'b0;
else
Q <= D;
end
endmodule

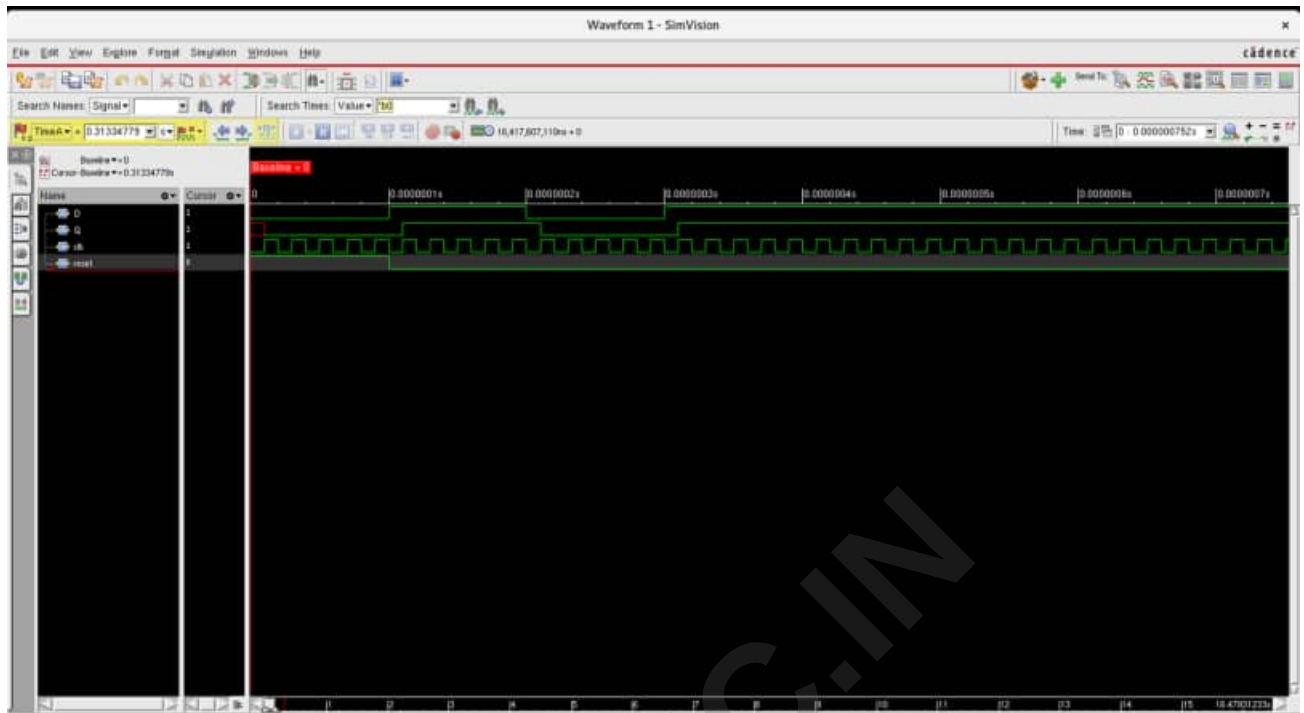
```

Testbench Code:

```

module tb_DFF();
reg D;
reg clk;
reg reset;
wire Q;
RisingEdge_DFlipFlop_SyncReset dut(D,clk,reset,Q);
initial begin
clk=0;
forever #10 clk = ~clk;
end
initial begin
reset=1;
D <= 0;
#100;
reset=0;
D <= 1;
#100;
D <= 0;
#100;
D <= 1;
end
endmodule

```

Output:**Waveform:****Timing Report:**

```

@genus:root: 8> report_timing -unconstrained
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s002.1
Generated on:      Apr 02 2023 01:20:42 pm
Module:           RisingEdge_DFlipFlop_SyncReset
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

Path 1: UNCONSTRAINED Setup Check with Pin Q_reg/CK->D
Startpoint: (R) D
Endpoint: (R) Q_reg/D

      Capture      Launch
Drv Adjust:++      0      0

      Setup:-      223
      Data Path:-  103

#-----#
# Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival Instance
#              (ff) (ps) (ps) (ps) (ps) Location
#-----#
D              -      -   R   (arrival)  1  1.7  0  0  0  (-,-)
q4_2398/Y     -      -   R   NOR2BXL  1  1.6  70 103 103  (-,-)
Q_reg/D       -      -   R   DFFQXL  1  -   -  0 103  (-,-)
#-----#

@genus:root: 9> report_area

```

Area Report:

```

@genus:root: 7> report_timing -unconstrained

=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025 02:13:39 pm
Module:           sr_flipflop
Operating conditions: slow (balanced tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

Path 1: UNCONSTRAINED
  Startpoint: (R) qbar_reg/CK
  Endpoint: (F) qbar

  Data Path:- 272

#-----
# Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival Instance
#              (fF) (ps) (ps) (ps) Location
#-----
qbar_reg/CK    -      -   R   (arrival)  2      -   0    0    0    (-,-)
qbar_reg/Q     -      CK->Q F   DFFQXL   1 0.0   26   272  272  (-,-)
qbar           -      -   F   (port)    -      -   -    0   272  (-,-)
#-----

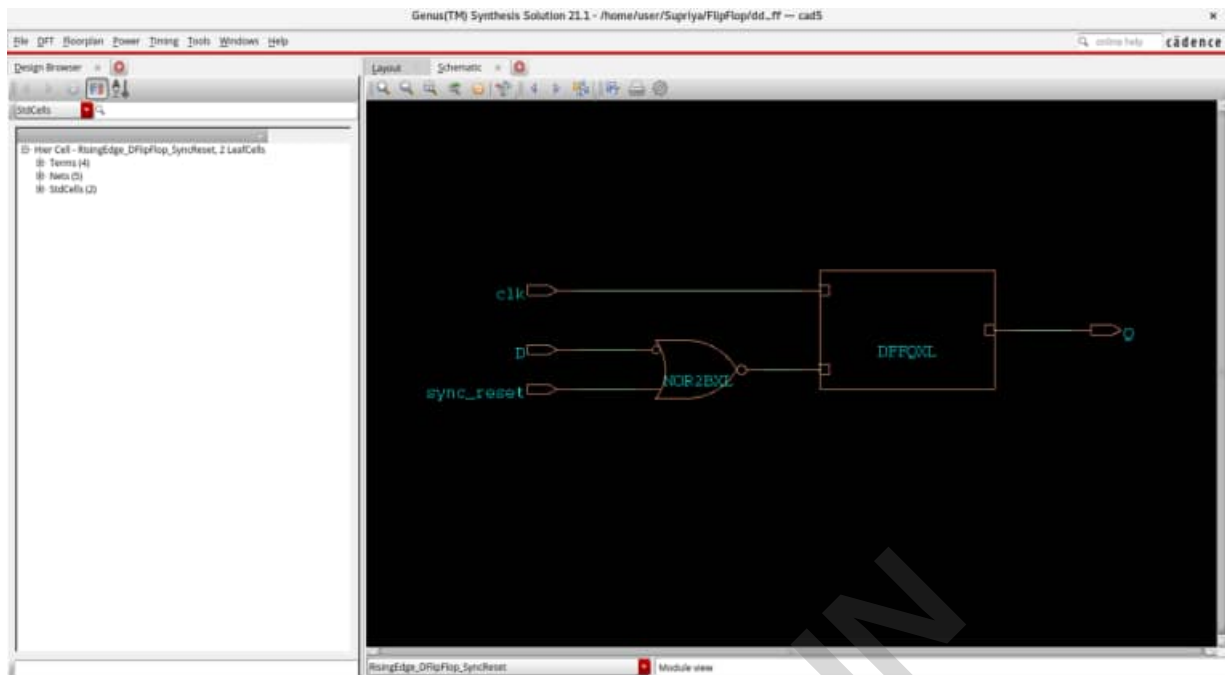
@genus:root: 8>

```

Power Report:

Power Details Report — cad5					
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1 (Jun 23 2022 14:32:08)					
Generated on: Apr 02 2025 13:26:07					
Module: design:RisingEdge_DFlipFlop_SyncReset					
Technology library: slow					
Operating conditions: slow (balanced_tree)					
Wireload mode: enclosed					
Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
Rising...cReset	2	108.770	368.415	53.460	421.875

RTL Schematic:



Power Requirements

- **Leakage Power:** 108.770nW
- **Internal Power:** 368.415nW
- **Net Power:** 53.460nW
- **Switching Power:** 421.875nW

Area Requirements

- **Cell Count:** 2
- **Cell Area:** 34.060
- **Net Area:** 0.00
- **Total Area:** 34.060

Timing and Critical Path

- **Maximum Delay:** 103 ps
- **Critical Path Delay:** 103 ps
- **Critical Path:** Q_2390/Y → DFF0XL (via NOR2BX1)

SR- Flipflop:

Aim : For SR- Flipflop.

- Write Verilog description
- Verify functionality using Test-bench
- Synthesize the design targeting suitable library and by setting area and timing constraints
- Tabulate the Area, Power and Delay for the Synthesized netlist
- Identify Critical path

Theory: An SR Flip-Flop, also known as a Set-Reset Flip-Flop, is one of the fundamental building blocks of sequential logic circuits. It is a bistable device, meaning it has two stable states and can store one bit of binary information (either 0 or 1). The flip-flop has two inputs, labeled S (Set) and R (Reset), along with two outputs Q and $\bar{Q}$ (Q bar), where $\bar{Q}$ is the complement of Q. The operation of the SR Flip-Flop is straightforward: when the Set input (S) is activated (logic 1) and Reset (R) is at logic 0, the output Q is set to 1. Conversely, when R is 1 and S is 0, the output Q is reset to 0. If both S and R are at logic 0, the flip-flop maintains its previous state, functioning as a memory element. However, if both inputs are set to logic 1 simultaneously, it creates an invalid or undefined condition, especially in the NOR gate-based implementation, as both outputs Q and $\bar{Q}$ would become 0, violating the rule of being complementary.

The SR Flip-Flop can be built using NOR or NAND gates, and it can operate either asynchronously, without a clock signal, or synchronously, with a clock input that controls the state change. Due to its simplicity, the SR Flip-Flop is widely used in applications such as data storage, control systems, switch debouncing, and small-scale memory designs. Despite its limitations, particularly the undefined state when both inputs are high, it serves as the foundation for more complex flip-flops like D, JK, and T flip-flops.

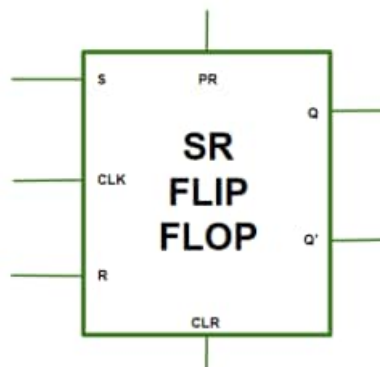


Fig.2.1 SR Flipflop Block Diagram.

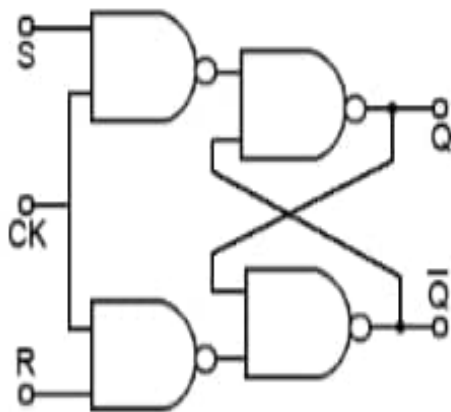


Fig. 2.2 SR-Flipflop using NAND gate.

Truth table:

S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

Verilog code:

```

module sr_flipflop(
    output reg q, qbar,
    input s, r, clk
);
    always @(posedge clk) begin
        if (s == 1 && r == 0) begin
            q <= 1;
            qbar <= 0;
        end
        else if (s == 0 && r == 1) begin
            q <= 0;
            qbar <= 1;
        end
        else if (s == 0 && r == 0) begin
            q <= q; // Hold previous state
            qbar <= qbar;
        end
        else if (s == 1 && r == 1) begin
            q <= 1'bx; // Undefined state
            qbar <= 1'bx;
        end
    end
end
endmodule

```


Testbench Code:

```

module sr_flipflop_tb;
    reg s, r, clk;
    wire q, qbar;

    // Instantiate the SR Flip-Flop
    sr_flipflop uut (.q(q), .qbar(qbar), .s(s), .r(r), .clk(clk));

    initial begin
        // Initialize inputs
        clk = 0; s = 0; r = 0;

        #10 s = 1; r = 0; // Set (Q = 1)
        #10 s = 0; r = 0; // Hold state
        #10 s = 0; r = 1; // Reset (Q = 0)
        #10 s = 1; r = 1; // Invalid state (Q = X)
        #10 s = 0; r = 0; // Hold state

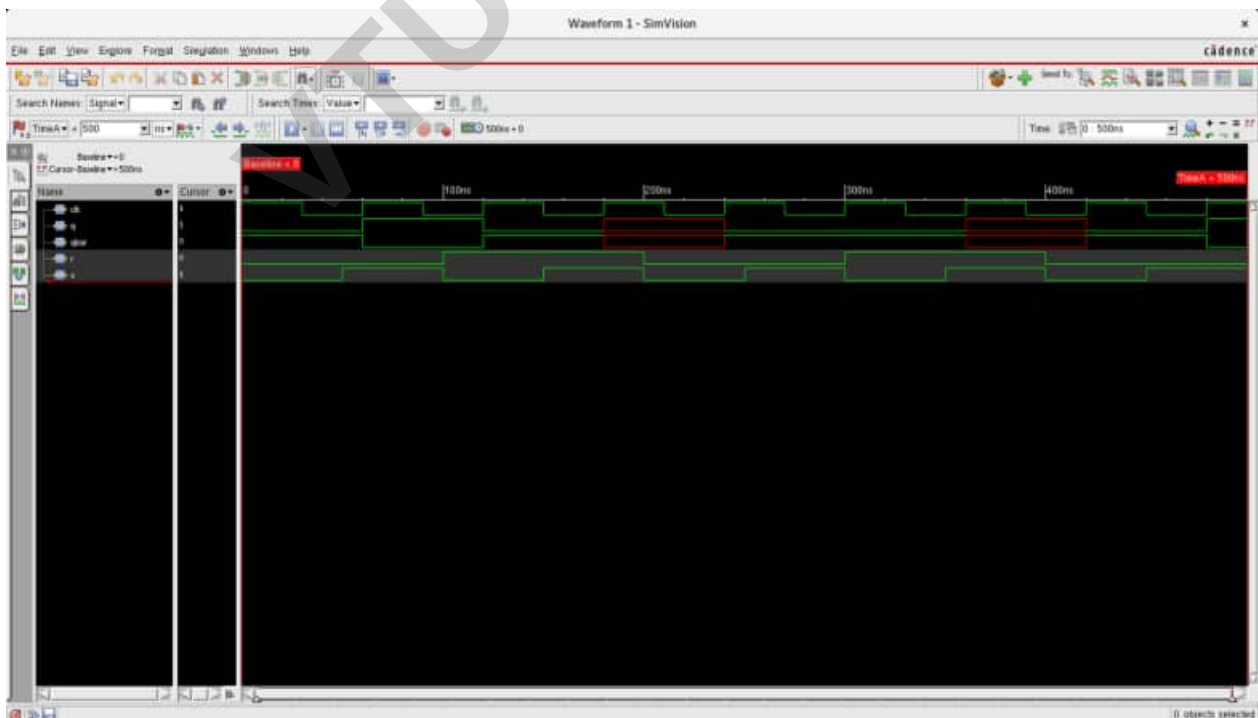
        #50 $finish; // End simulation
    end

    // Clock Generation
    always #5 clk = ~clk; // Toggle clock every 5 time units
endmodule

```

Output:

Waveform:



Timing Report:

```
@genus:root: 7> report_timing -unconstrained
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025  02:13:39 pm
Module:            sr_flipflop
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Path 1: UNCONSTRAINED
Startpoint: (R) qbar_reg/CK
Endpoint: (F) qbar

Data Path:- 272

#-----#
# Timing Point  Flags  Arc  Edge  Cell  Fanout Load Trans Delay Arrival Instance
#                                     (fF) (ps) (ps) (ps) Location
#-----#
# qbar_reg/CK    -      -   R   (arrival)  2      -    0    0    0    (-,-)
# qbar_reg/Q      -      CK->Q F   DFF0XL    1  0.0   26   272   272   (-,-)
# qbar            -      -   F   (port)      -    -    -    0   272   (-,-)
#-----#

@genus:root: 8> 
```

Area Report:

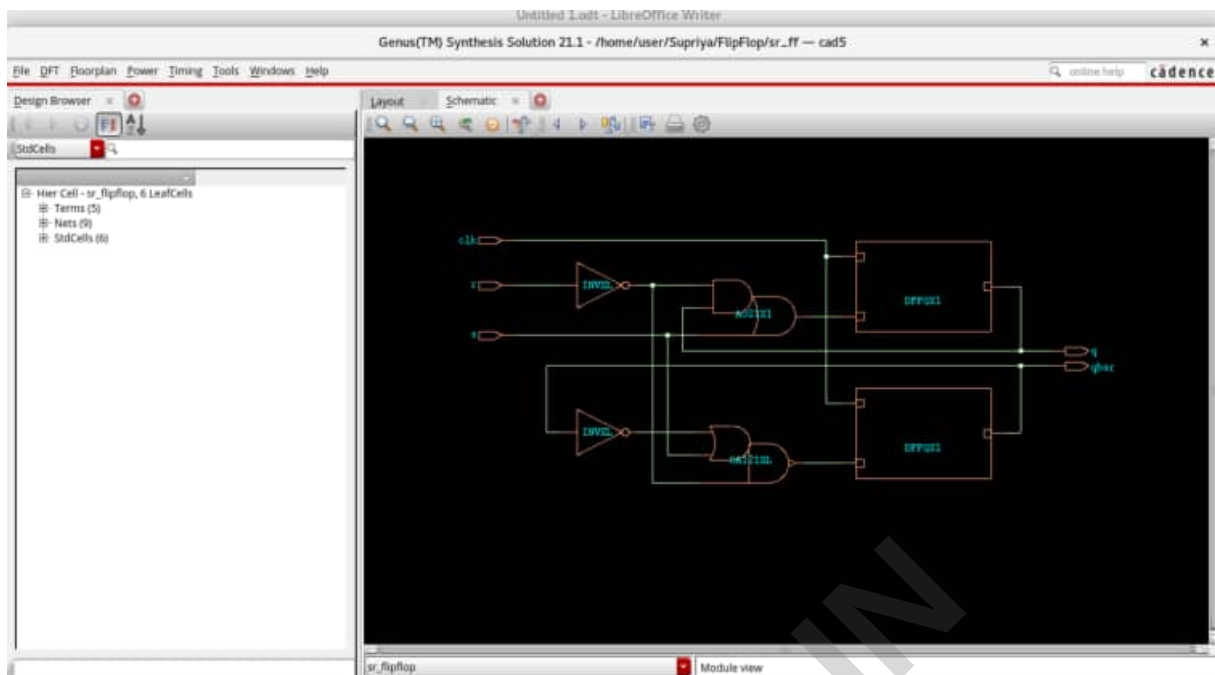
```
@genus:root: 8> report_area
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025  02:14:52 pm
Module:            sr_flipflop
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Instance Module Cell Count Cell Area Net Area Total Area Wireload
-----
sr_flipflop 3 34.060 0.000 34.060 <none> (D)
(D) = wireload is default in technology library
@genus:root: 9> 
```

Power Report:

Power Details Report — cad5					
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1 (Jun 23 2022 14:32:08)					
Generated on: Apr 02 2025 14:40:03					
Module: design:sr_flipflop					
Technology library: slow					
Operating conditions: slow (balanced_tree)					
Wireload mode: enclosed					
Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
sr_flipflop	6	266.650	824.576	144.180	968.756

RTL Schematic:



Power Requirements

- **Leakage Power:** 266.650 nW
- **Internal Power:** 824.576nW
- **Net Power:** 144.180nW
- **Switching Power:** 968.756nW

Area Requirements

- **Cell Count:** 3
- **Cell Area:** 34.060
- **Net Area:** 0.00
- **Total Area:** 34.060

Timing and Critical Path

- **Maximum Delay:** 272 ps
- **Critical Path Delay:** 272 ps
- **Critical Path:** qbar_reg/CK → DFF0XL → qbar (port)

JK Flipflop

Aim: For JK- Flipflop.

- Write Verilog description
- Verify functionality using Test-bench
- Synthesize the design targeting suitable library and by setting area and timing constraints
- Tabulate the Area, Power and Delay for the Synthesized netlist
- Identify Critical path

Theory:

The **JK Flip-Flop** is an advanced and versatile type of flip-flop that eliminates the invalid state encountered in the SR Flip-Flop. Named after its inventor **Jack Kilby**, the JK Flip-Flop has two inputs, labeled **J** and **K**, along with two outputs **Q** and $\bar{Q}$. It also operates based on a **clock signal**, making it a **synchronous sequential circuit**. The JK Flip-Flop functions similarly to the SR Flip-Flop in most input conditions: when $J = 1$ and $K = 0$, it sets the output Q to 1; when $J = 0$ and $K = 1$, it resets Q to 0; and when both J and K are 0, the output retains its previous state (no change). However, the key advantage of the JK Flip-Flop is its behavior when both inputs J and K are set to 1 — instead of producing an invalid state like in SR Flip-Flop, the JK Flip-Flop toggles its output. That means if the current state of Q is 1, it becomes 0, and vice versa.

This toggle feature makes the JK Flip-Flop highly useful in **counters, frequency dividers, and control applications**. It is considered a **universal flip-flop** because it can be configured to behave like an SR, D, or T flip-flop by appropriately connecting its inputs. Internally, it typically consists of logic gates arranged in a feedback configuration that ensures stable operation based on the clock edge. The JK Flip-Flop is triggered by either the **rising or falling edge** of the clock, depending on the design. Overall, its versatility, elimination of the invalid state, and reliable toggle operation make the JK Flip-Flop a fundamental component in digital system design.

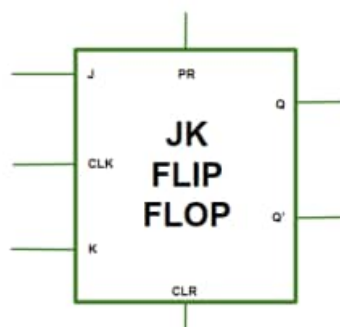
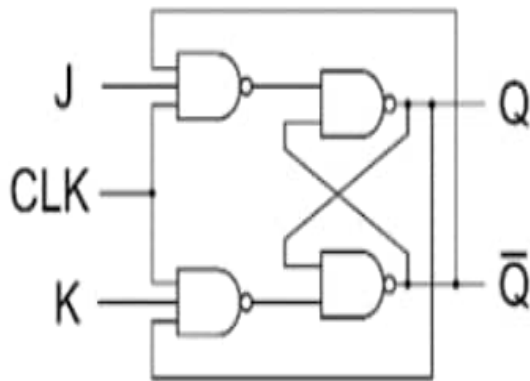


Fig.3.1 Block Diagram of JK Flipflop

Truth table:

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Fig. 3.2 JK Flipflop using NAND gate.**Verilog Code:**

```

module jkff(input reset, input clk, input j, input k, output reg q, output qnot);
assign qnot=~q;
always @(posedge clk)
  if (reset) q<=1'b0; else
    case ({j, k})
      2'b00: q<=q;
      2'b01: q<=1'b0;
      2'b10: q<=1'b1;
      2'b11: q<=~q;
    endcase
endmodule

```

Testbench Code:

```

module test;
reg clk=0;
reg j=0;
reg k=0;
reg reset=0;
wire q, qnot;
jkff dut(reset, clk,j,k,q,qnot);

initial
  begin

    #10 j=1'b0; k=1'b0;
    #10 j=1'b1; k=1'b0;
    #10 j=1'b0; k=1'b1;
    #10 j=1'b1; k=1'b1;
  end

```

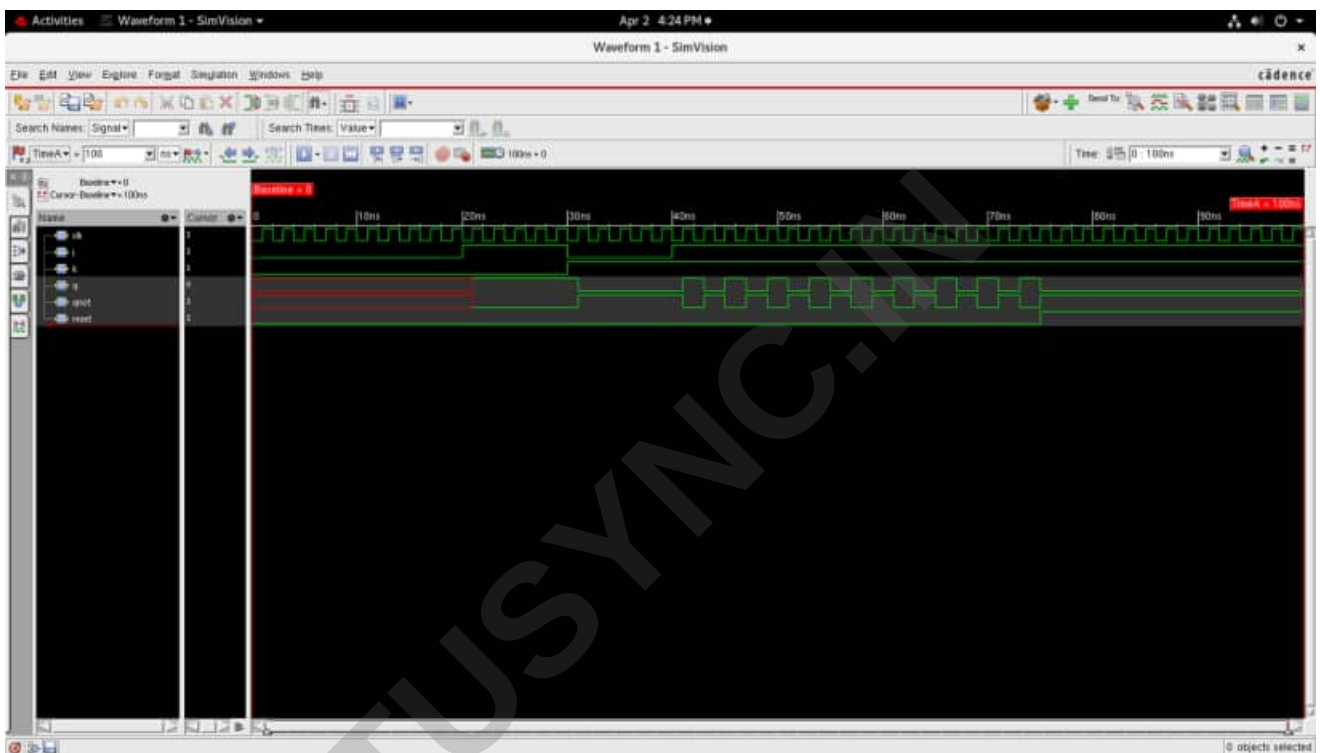
```

#10 j=1'b1; k=1'b1; // set your JK here
#25 reset=1'b1;
#25 $finish;
end
always #1 clk=~clk;
endmodule

```

Output:

Waveform:



Timing Report:

```

@genus:root: 11> report_timing -unconstrained
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025 04:30:40 pm
Module:            jkff
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Path 1: UNCONSTRAINED Setup Check with Pin q_reg/CK->D
Startpoint: (R) q_reg/CK
Endpoint: (R) q_reg/D

      Setup:- 226
      Data Path:- 508

#-----#
# Timing Point   Flags   Arc   Edge   Cell          Fanout Load Trans Delay Arrival Instance
#-----#-----#-----#-----#-----#-----#-----#
#               (arrival)
q_reg/CK         -       -       R       DFFQX1         1       -       0       0       0       (-,-)
q_reg/Q          -       CK->Q   R       DFFQX1         2       1.6     41      242      242      (-,-)
g207/Y           -       A->Y   F       INVXL          2       3.7     55       53      295      (-,-)
g206_5107/Y      -       S0->Y   F       MXI2XL         1       1.7     87      129      424      (-,-)
g205_2398/Y      -       B->Y   R       NOR2XL         1       1.6     80       84      508      (-,-)
q_reg/D          -       -       R       DFFQX1         1       -       -       0       508      (-,-)
#-----#

@genus:root: 12> 

```

Area Report:

```
@genus:root: 12> report_area
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025  04:31:33 pm
Module:            jkff
Operating conditions:  slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Instance Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
jkff              5      29.519    0.000     29.519 <none> (D)
(D) = wireload is default in technology library
@genus:root: 13> 
```

Power Report

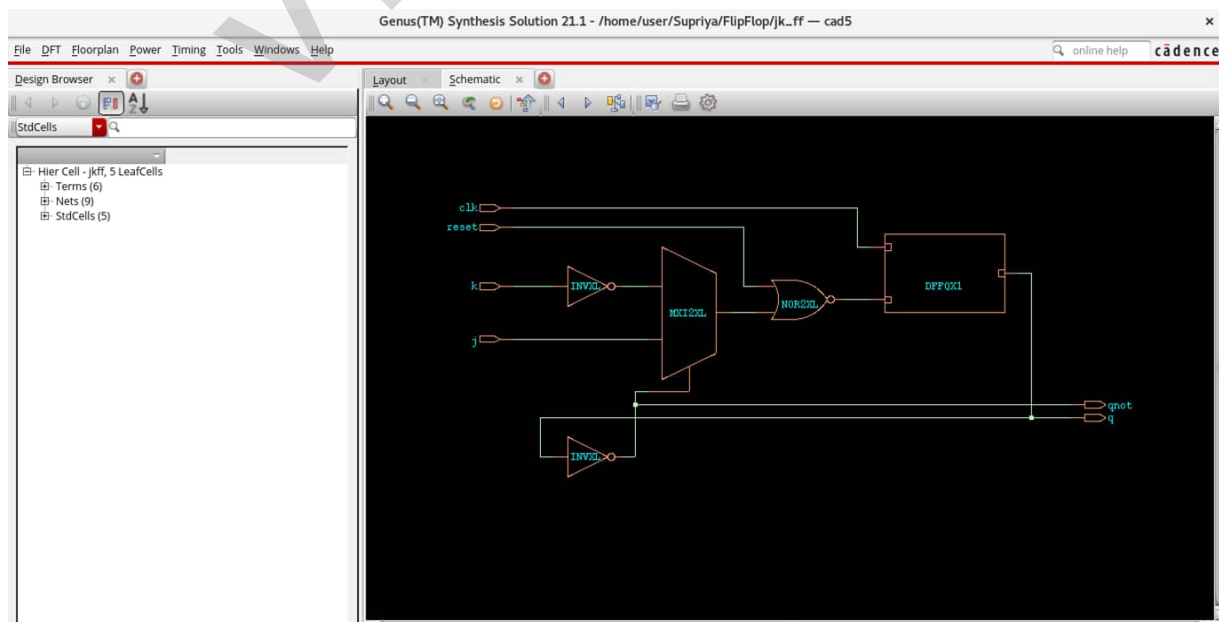
Power Details Report — cad5

Generated by: Genus(TM) Synthesis Solution 21.14-s082_1 (Jun 23 2022 14:32:08)
 Generated on: Apr 02 2025 16:34:41
 Module: design:jkff
 Technology library: slow
 Operating conditions: slow (balanced_tree)
 Wireload mode: enclosed

Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
jkff	5	155.795	506.098	119.677	625.776

Close Help

RTL Schematic:



Power Requirements

- **Leakage Power:** 155.795 nW
- **Internal Power:** 506.098 nW
- **Net Power:** 119.670 nW
- **Switching Power:** 625.667 nW

Area Requirements

- **Cell Count:** 5
- **Cell Area:** 29.519
- **Net Area:** 0.00
- **Total Area:** 29.519

Timing and Critical Path

- **Maximum Delay:** 508 ps
- **Critical Path Delay:** 508 ps
- **Critical Path:** $q_reg/CK \rightarrow DFF0XL \rightarrow INVX1XL \rightarrow INVX1XL \rightarrow NOR2X2L \rightarrow DFF0XL$

Experiment 5: MOD N Counter

Aim: To design a four bit Synchronous MOD-N counter with Asynchronous reset

- Write Verilog Code
- Verify functionality using Test-bench
- Synthesize the design targeting suitable library and by setting area and timing constraints
- Tabulate the Area, Power and Delay for the Synthesized netlist Identify Critical path
- Verify the functionality using Gate level netlist and compare the results at RTL and gate level netlist.

Theory:

A **MOD-N counter** is a type of sequential counter that counts a specific number of clock pulses before resetting to zero. The “N” in MOD-N refers to the number of unique states the counter goes through. For example, a MOD-4 counter counts from 0 to 3, then resets back to 0 on the next clock pulse.

MOD-N counters can be built using flip-flops, with the number of flip-flops determining the maximum count value. A 4-bit counter, for instance, can count up to $2^4 = 16$ states, so the maximum modulus it can support is 16. To create a MOD-N counter (where $N \leq 16$), logic is added to reset the counter once it reaches $N-1$.

There are two main types of counters:

- **Synchronous Counter:** All flip-flops are triggered by the same clock signal, leading to faster and more reliable operation.
- **Asynchronous Counter:** Flip-flops are triggered one after another, causing propagation delay and timing issues.

The MOD-N counter is widely used in digital systems for applications like frequency division, event counting, and state machine sequencing. Its flexibility and predictable behavior make it an essential component in digital electronics.

Verilog Code:

```
module mod_n_counter(
    input clk,           // Clock signal
    input rst_n,         // Asynchronous reset (active low)
    input [3:0] N,       // Modulus value (up to 15)
    output reg [3:0] count // 4-bit counter output
);

    always @(posedge clk or negedge rst_n) begin
        if (~rst_n) // Active low reset
            count <= 4'b0000; // Reset counter to 0
        else if (count == N-1)
            count <= 4'b0000;
    end
endmodule
```

```

        count <= 4'b0000; // Reset counter when it reaches N-1
    else
        count <= count + 1; // Increment counter
    end

```

Testbench Code:

```

module tb_mod_n_counter;

    reg clk; // Clock signal
    reg rst_n; // Asynchronous reset (active low)
    reg [3:0] N; // Modulus value
    wire [3:0] count; // Counter output

    // Instantiate the mod_n_counter module
    mod_n_counter uut (
        .clk(clk),
        .rst_n(rst_n),
        .N(N),
        .count(count)
    );

    // Clock generation
    always begin
        #5 clk = ~clk; // Toggle clock every 5 time units
    end

    // Test procedure
    initial begin
        clk = 0; // Initialize clock
        rst_n = 0; // Apply reset
        N = 4; // Set modulus to 4

        // Apply reset
        #10 rst_n = 1; // Release reset after 10 time units

        // Test mod 4 counter
        #40;

        // Change modulus to 6
        N = 6;
        #40;

        // Test mod 8 counter
        N = 8;
        #80;
    end

```

```

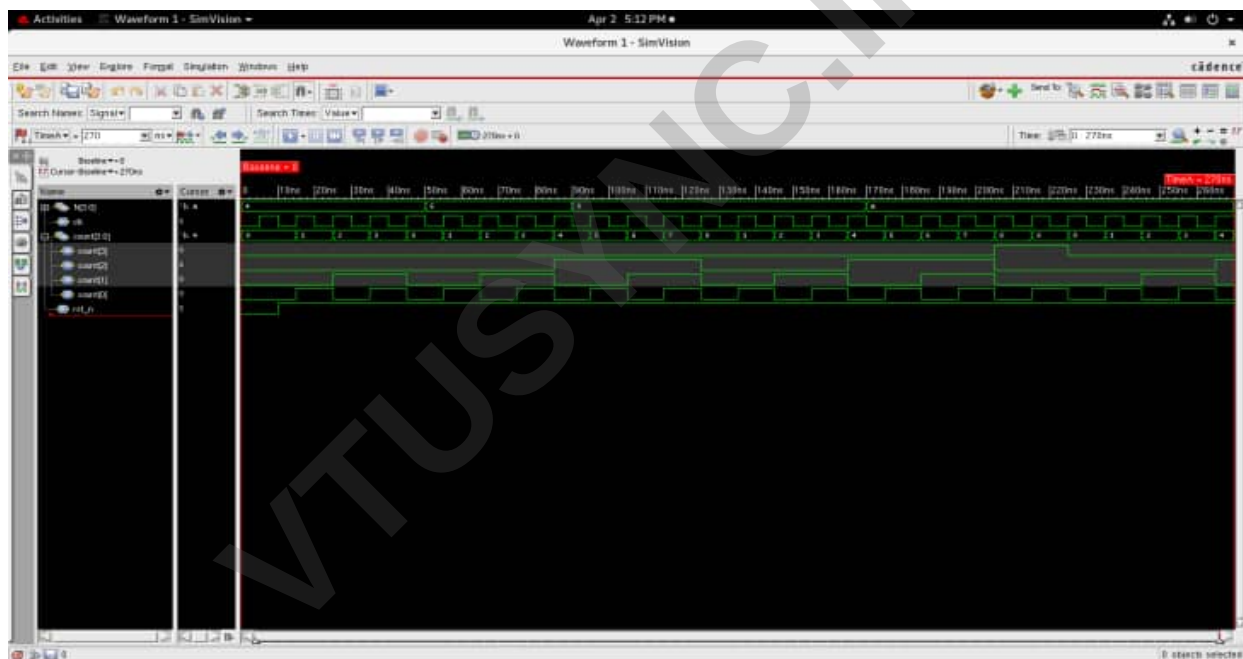
// Test mod 10 counter
N = 10;
#100;

// End simulation
$finish;
end
// Monitor the output
initial begin
    $monitor("Time=%0t, clk=%b, rst_n=%b, N=%d, count=%d", $time, clk, rst_n, N,
count);
    end
endmodule

```

Output:

Waveform:



Timing Report:

```
@genus:root: 9> report_timing -unconstrained
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025 05:01:12 pm
Module:           mod_n_counter
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

Path 1: UNCONSTRAINED Setup Check with Pin count_reg[2]/CK->D
Startpoint: (R) count_reg[2]/CK
Endpoint: (F) count_reg[2]/D

      Setup:- 160
      Data Path:- 1190

#-----#
# Timing Point  Flags    Arc    Edge    Cell      Fanout Load Trans Delay Arrival Instance
#                                     (fF)  (ps)  (ps)  (ps)  Location
#-----#
count_reg[2]/CK -      -      R      (arrival)  4      -      0      0      0      (-,-)
count_reg[2]/QN -      CK->QN F      DFFRX1     4  6.6    69    416    416    (-,-)
g371_7482/Y -      B->Y  R      NOR2XL     3  6.3    211   172    589    (-,-)
g367_8246/Y -      B0->Y F      AOI21XL    1  3.5    126    88    677    (-,-)
g358_6783/Y -      B->Y  F      XNOR2X1    1  1.7    52    148    825    (-,-)
g356_8428/Y -      B0->Y R      AOI211XL   4  6.8    367   294    1120   (-,-)
g354_2398/Y -      C0->Y F      AOI211XL   1  1.7    131    70    1190   (-,-)
count_reg[2]/D -      -      F      DFFRX1     1      -      -      0    1190   (-,-)
#-----#

@genus:root: 10> 
```

Area Report:

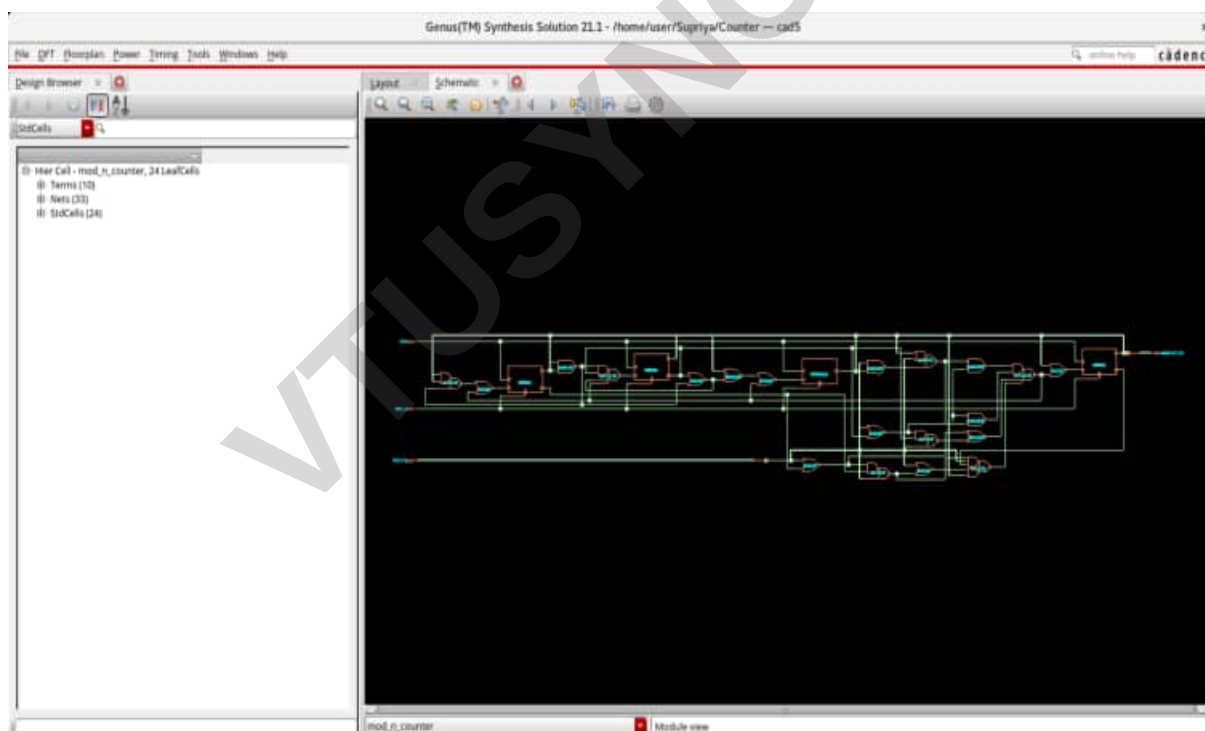
```
@genus:root: 10> report_area
=====
Generated by:      Genus(TM) Synthesis Solution 21.14-s082_1
Generated on:      Apr 02 2025 05:01:56 pm
Module:           mod_n_counter
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

Instance  Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
mod_n_counter      24    174.844    0.000    174.844 <none> (D)
(D) = wireload is default in technology library
@genus:root: 11> 
```

Power Report:

Power Details Report — cad5					
Generated by: Genus(TM) Synthesis Solution 21.14-s082_1 (Jun 23 2022 14:32:08)					
Generated on: Apr 02 2025 17:03:35					
Module: design:mod_n_counter					
Technology library: slow					
Operating conditions: slow (balanced_tree)					
Wireload mode: enclosed					
Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
mod_n_counter	24	874.796	2600.516	647.604	3248.120

RTL Schematic:



Power Requirements

- **Leakage Power:** 874.796 nW
- **Internal Power:** 2600.516 nW
- **Net Power:** 647.604 nW
- **Switching Power:** 3248.120 nW

Area Requirements

- **Cell Count:** 24
- **Cell Area:** 174.844
- **Net Area:** 0.00
- **Total Area:** 174.844

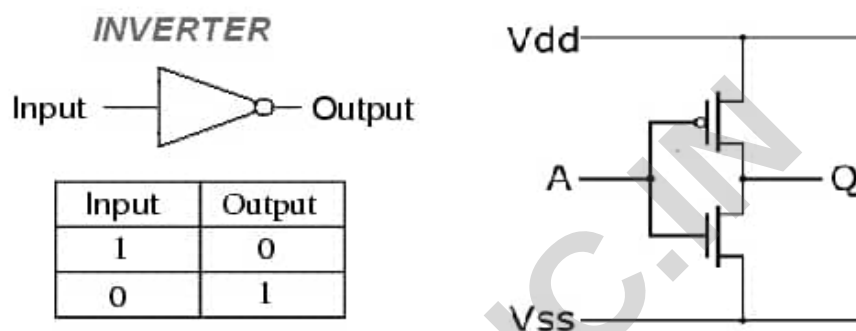
Timing and Critical Path

- **Maximum Delay:** 1190 ps
- **Critical Path Delay:** 1190 ps
- **Critical Path:** count_reg[2]/CK → g371_7482/Y (NOR2X2L)
 - g367_8246/Y (AOI21X1L)
 - g358_6783/Y (XNOR2X1)
 - g356_8428/Y (AOI21X1L)
 - g354_2398/Y (AOI21X1L)
 - count_reg[2]/D (DFFRX1)

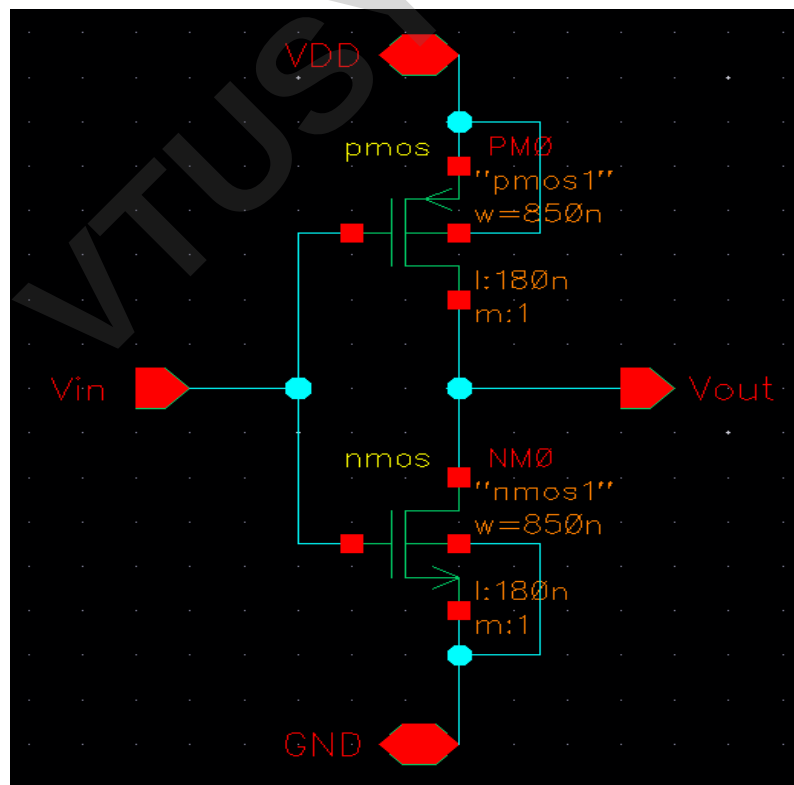
Experiment 6: CMOS Inverter

Aim: To Construct the schematic of CMOS inverter with load capacitance of 0.1pF and set the widths of inverter with $W_n = W_p$, $W_n = 2W_p$, $W_n = W_p/2$ and length at selected technology. Carry out the following: i. Set the input signal to a pulse with rise time, fall time of 1ns and pulse width of 10ns and the time period of 20ns and plot the input voltage and output voltage of designed inverter? ii. From the simulation result compute t_{pHL} , t_{pLH} and t_d for all three geometrical settings of width? iii. Tabulate the results of delay and find the best geometry for minimum delay for CMOS inverter. b) Draw layout of inverter with $W_p/W_n = 40/20$, use optimum layout methods. Verify for DRC and LVS, extract parasitic and perform post layout simulations, compare the results with pre layout simulations and compare the results.

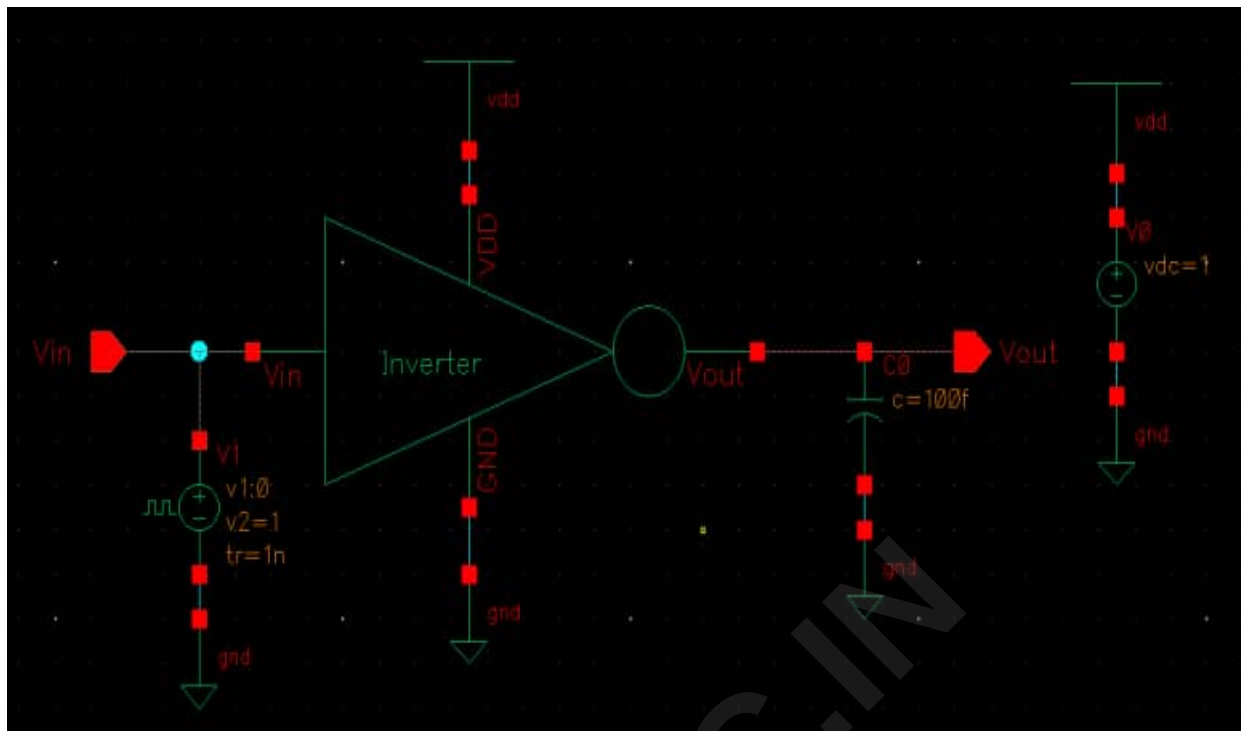
CMOS Inverter Symbol and Truth Table



Schematic of CMOS Inverter:



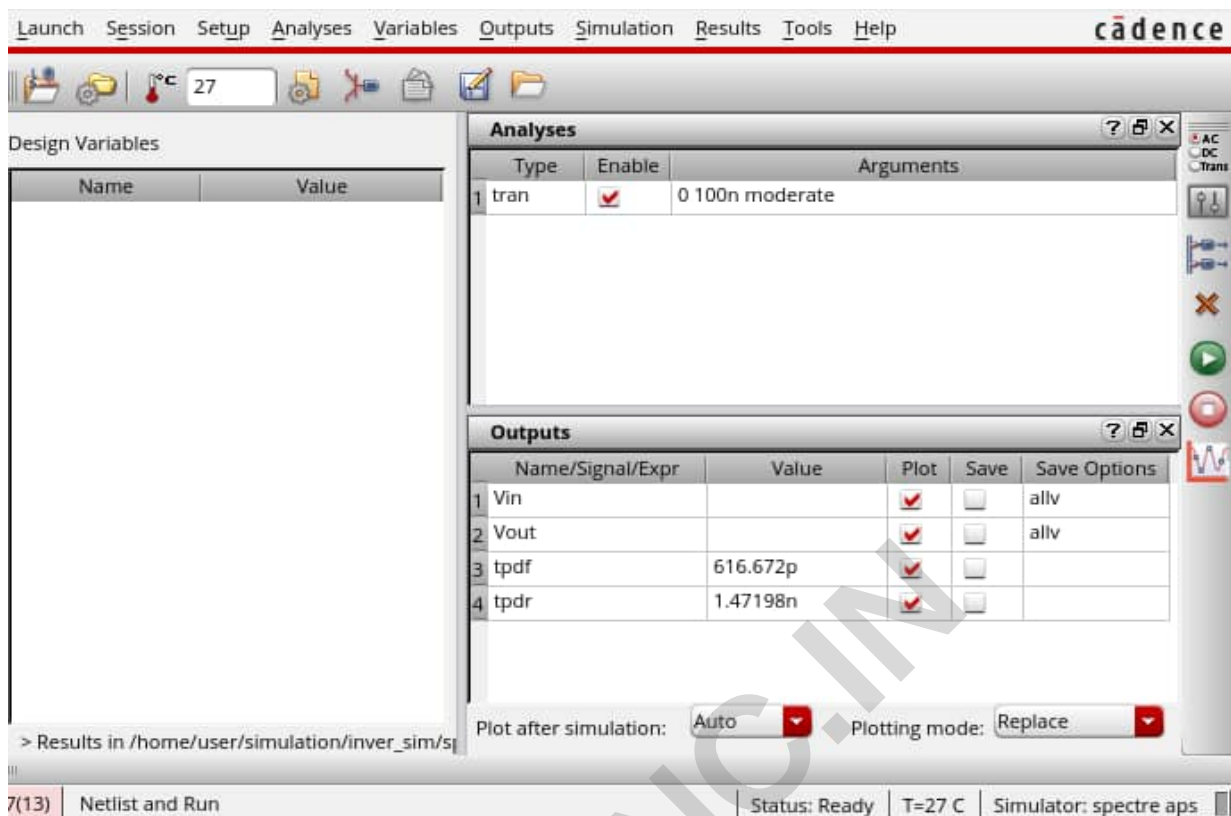
Symbol of CMOS Inverter



Simulation results: Transient and DC Characteristics



Rise time and Fall time Analysis when $W_n=W_p$

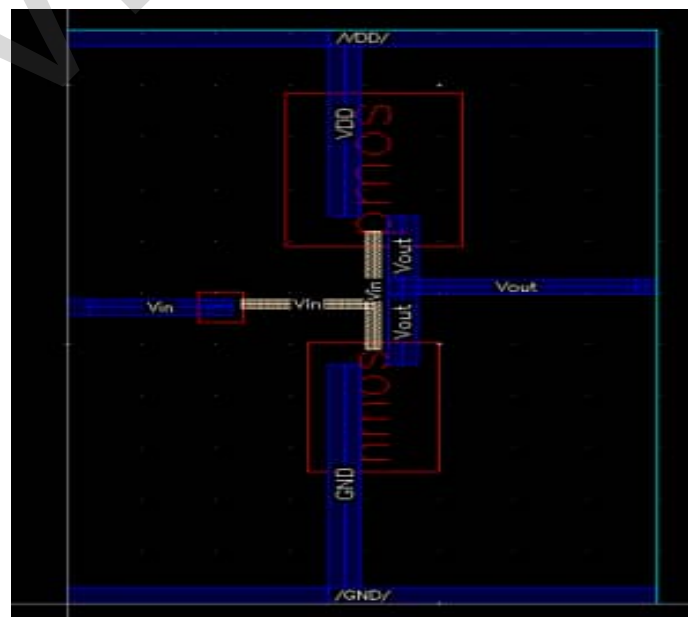


Fall Time t_f : 616.672p s

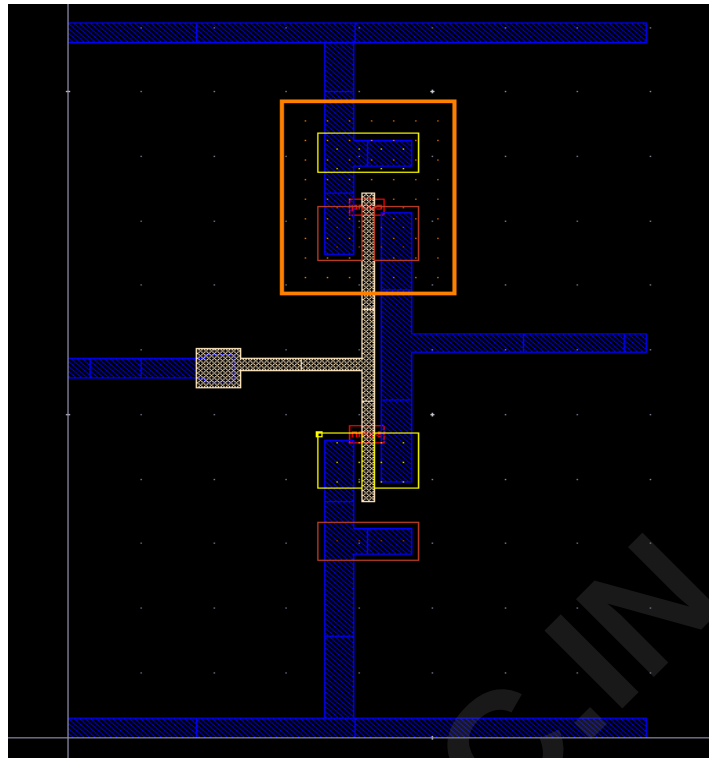
Rise Time t_r : 1.47198n s

Total Time delay t_d : $(t_f+t_r)/2 = 1.47198n$ s

Layout of CMOS Inverter:



av_extracted view of CMOS Inverter after parasitic extraction (RC):

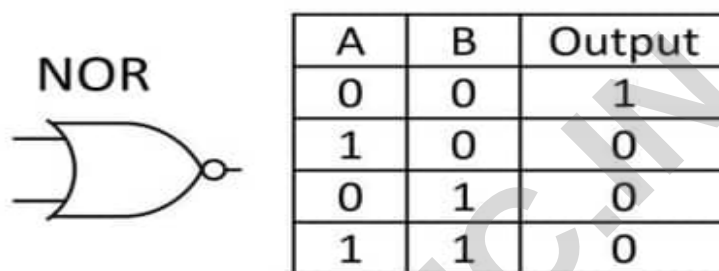


Experiment 7: CMOS NOR gate

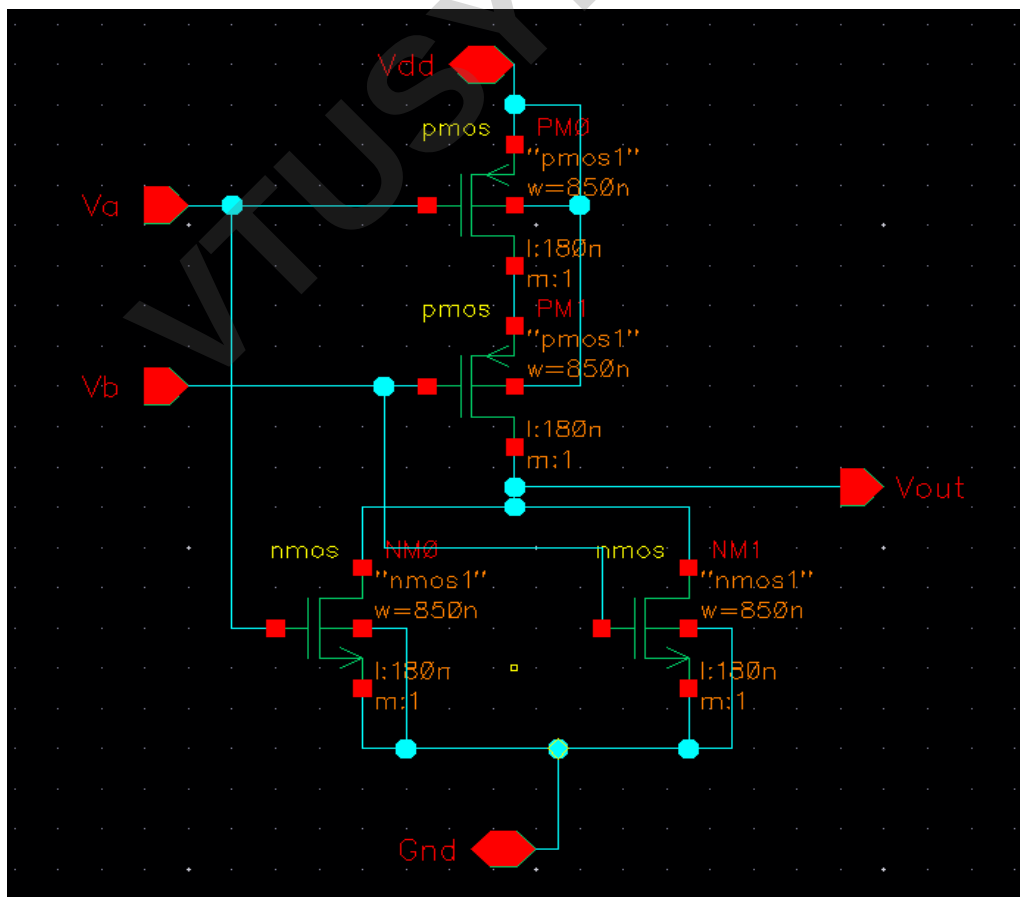
Aim: To capture the schematic of CMOS NOR gate with load capacitance of 0.1pF and set the widths of $W_n = W_p$, $W_n = 2W_p$, $W_n = W_p/2$ and length at selected technology. Carry out the following: i. Set the input signal to a pulse with rise time, fall time of 1ns and pulse width of 10ns and the time period of 20ns and plot the input voltage and output voltage of designed inverter? ii. From the simulation result compute t_{pHL} , t_{pLH} and t_d for all three geometrical settings of width? iii. Tabulate the delay results and find the best geometry for minimum delay for CMOS inverter.

NOR gate Symbol and Truth Table

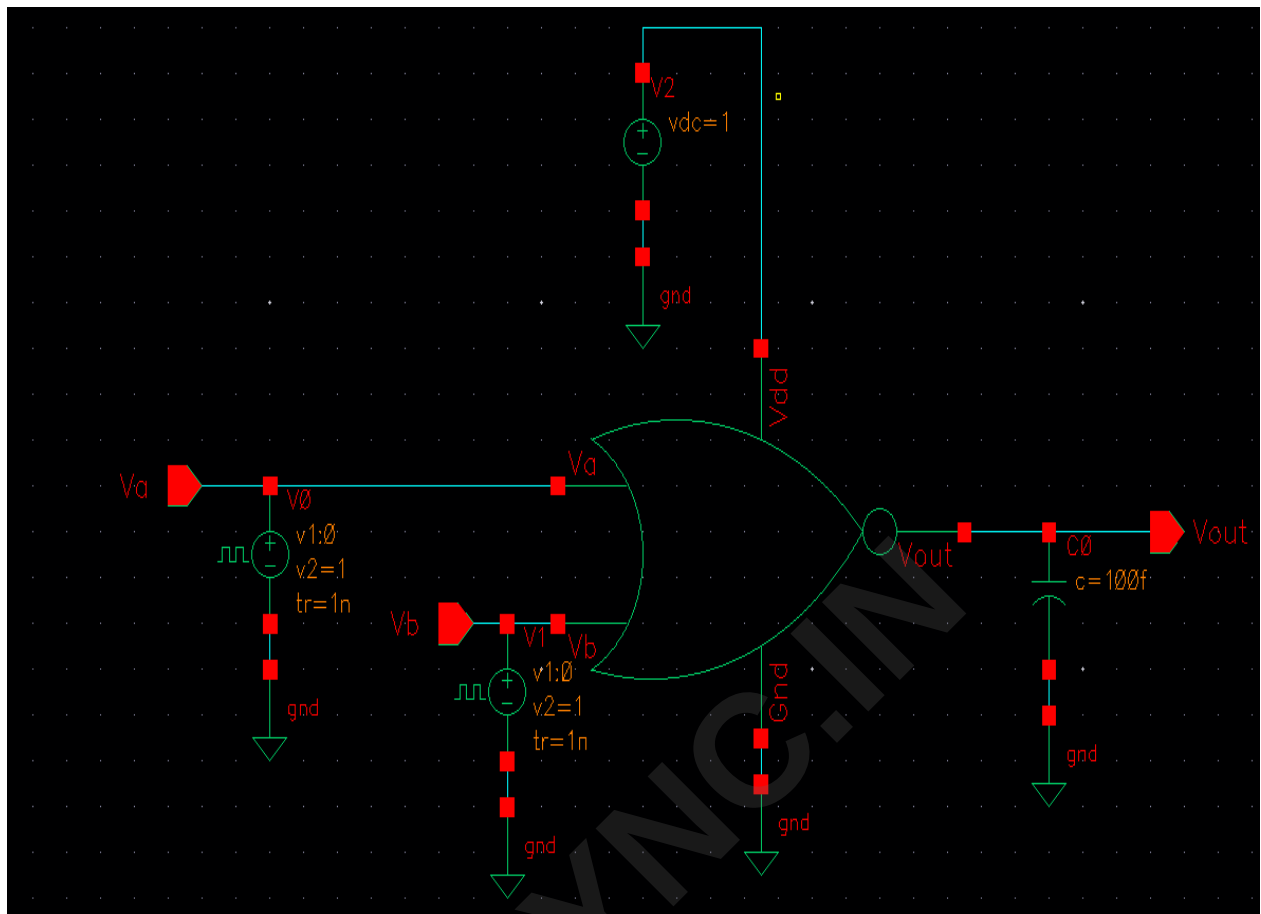
The NOR gate is a universal logic gate that can be built using only two transistors. It can be thought of as the not OR gate or having inverted logic to the OR gate.



Schematic of CMOS NOR gate:



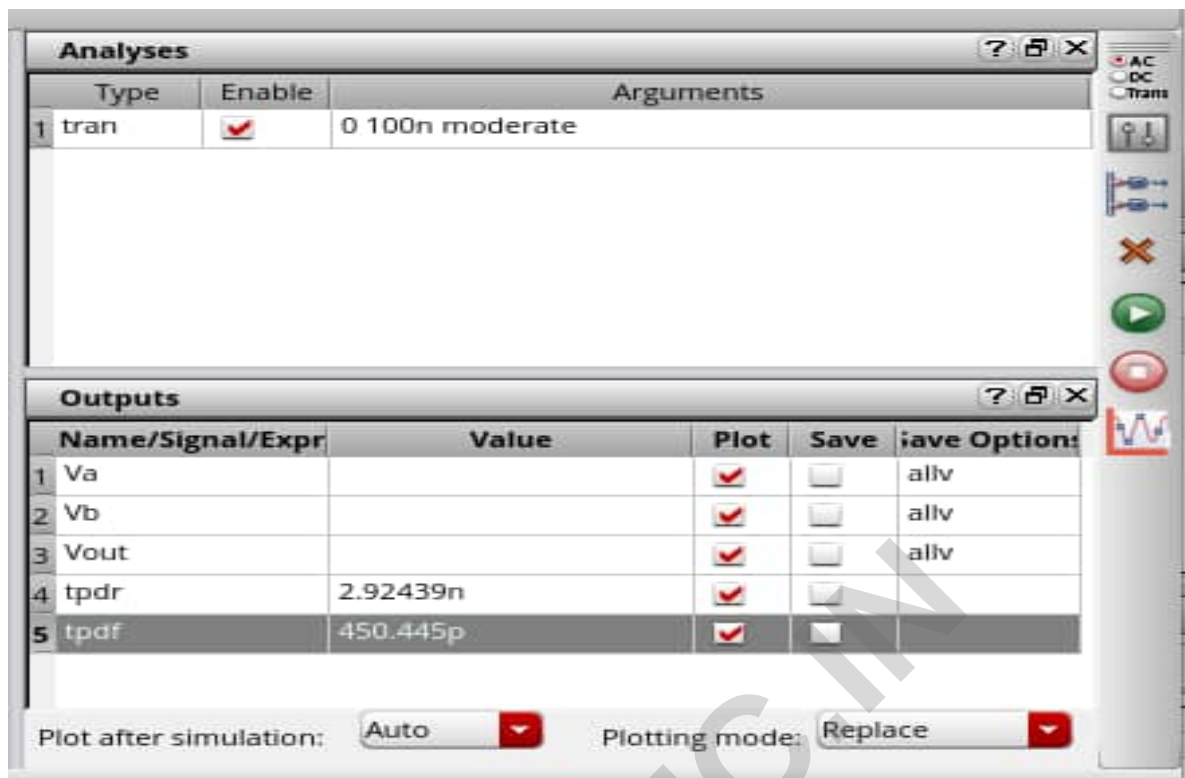
Symbol of CMOS Inverter



Simulation results: Transient and DC Characteristics



Rise time and Fall time Analysis when $W_n=W_p$



Fall Time t_f : 450.445p s

Rise Time t_r : 2.92439n s

Total Time delay t_d : $(t_f+t_r)/2 = 1.6874175$ n s

Experiment 8: Schematic of the Boolean Expression $Y = \overline{AB + CD + E}$

Aim: To Construct the schematic of the Boolean Expression $Y = \overline{AB + CD + E}$ using CMOS Logic. Verification of the functionality of the expression is also needed to find out the delay t_d for some combination of input vectors. Verification of the functionality of the expression is also requires input vectors. And tabulate the results.

Simplification of Boolean Equation $Y = \overline{AB + CD + E}$

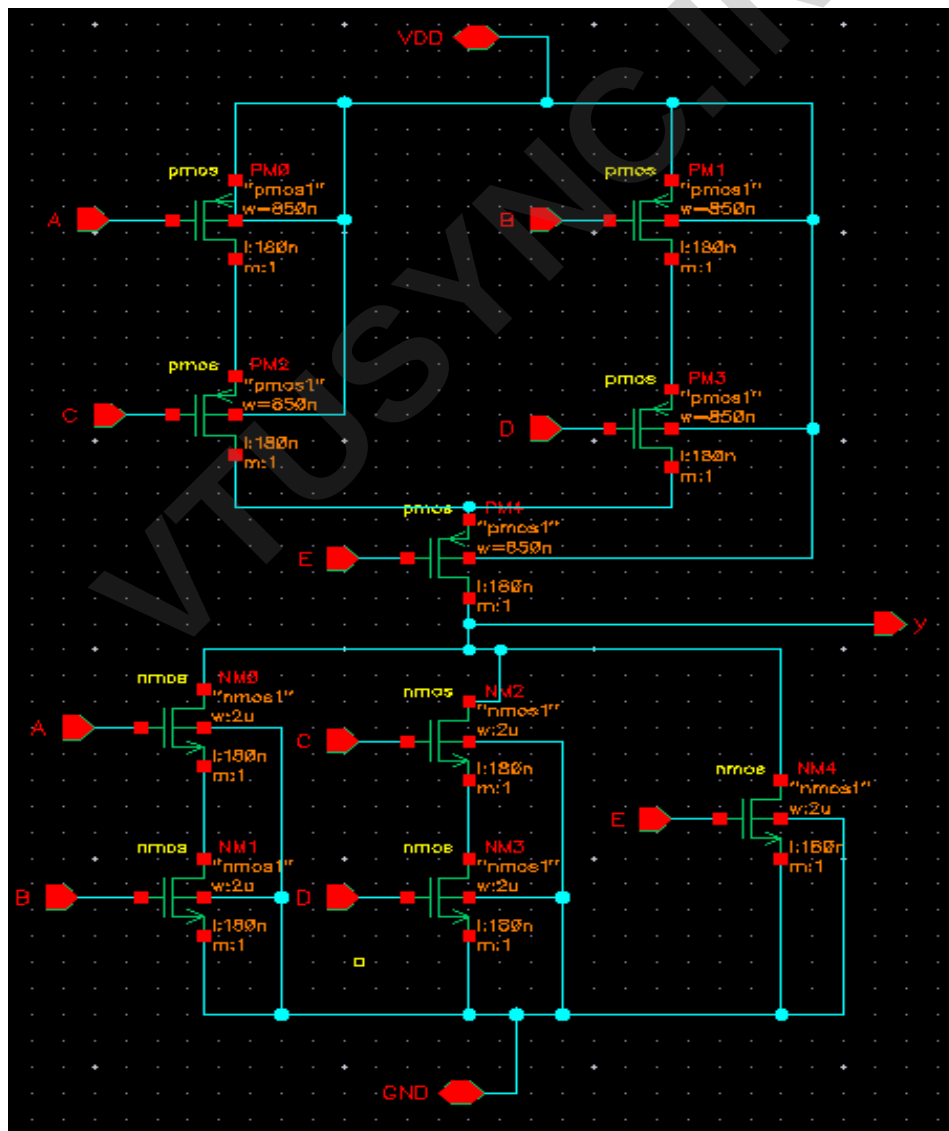
Apply Demorgan's Law to $Y = \overline{AB + CD + E}$

$$\rightarrow \overline{AB} \cdot \overline{CD} \cdot \overline{E}$$

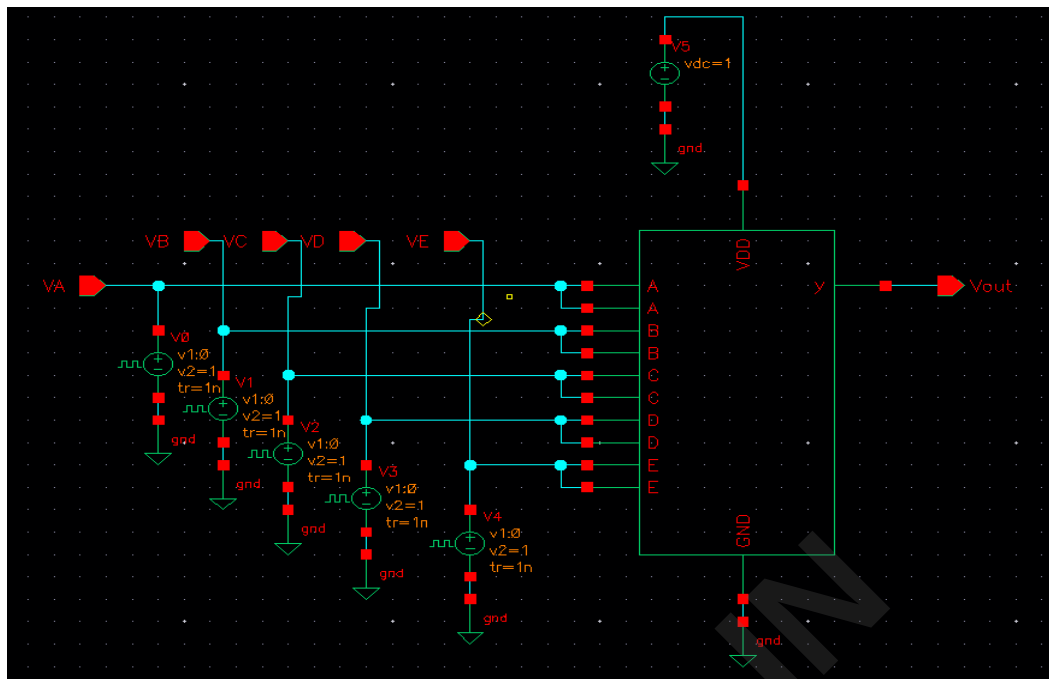
$$\rightarrow (\overline{A} + \overline{B}) \cdot (\overline{C} + \overline{D}) \cdot \overline{E}$$

The simplified Boolean expression is $Y = (\overline{A} + \overline{B}) \cdot (\overline{C} + \overline{D}) \cdot \overline{E}$

Schematic of Boolean equation $(\overline{A} + \overline{B}) \cdot (\overline{C} + \overline{D}) \cdot \overline{E}$



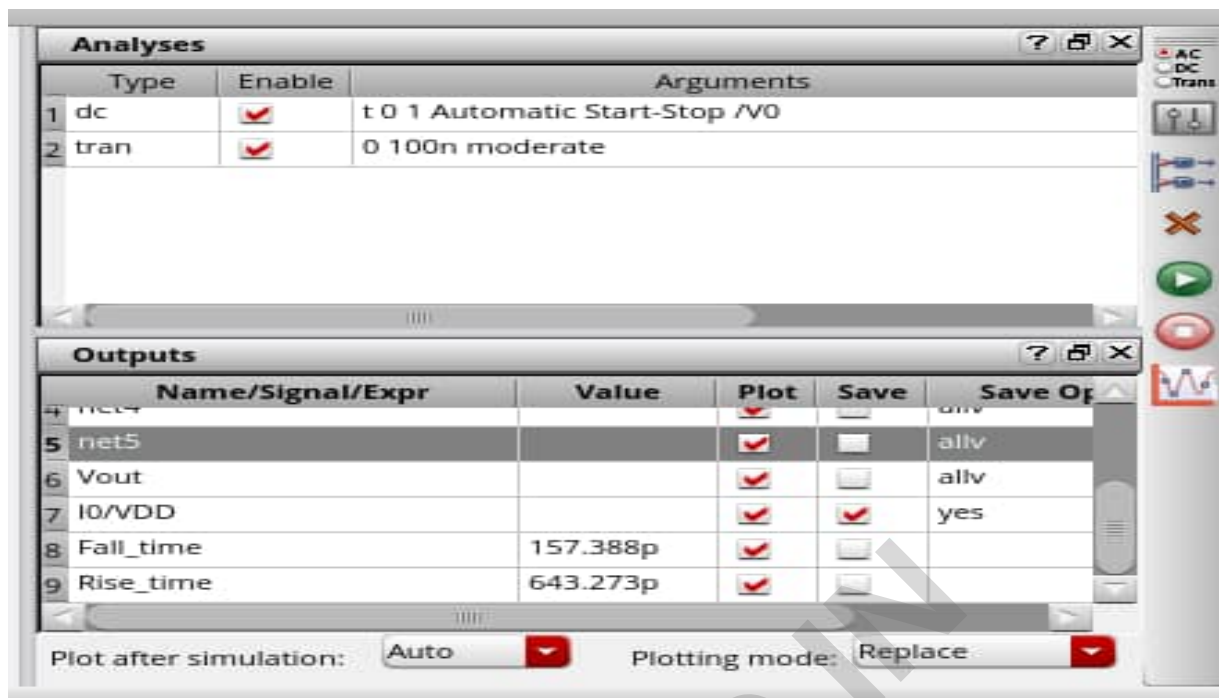
Symbolic representation of Boolean equation $(\bar{A} + \bar{B}) \cdot (\bar{C} + \bar{D}) \cdot \bar{E}$



Simulation results: Transient and DC Characteristics



Rise time and Fall time Analysis when $W_n=W_p$



Fall Time t_f : 157.388 p s

Rise Time t_r : 643.273 p s

Total Time delay t_d : $(t_f+t_r)/2 = 400.3305$ p s

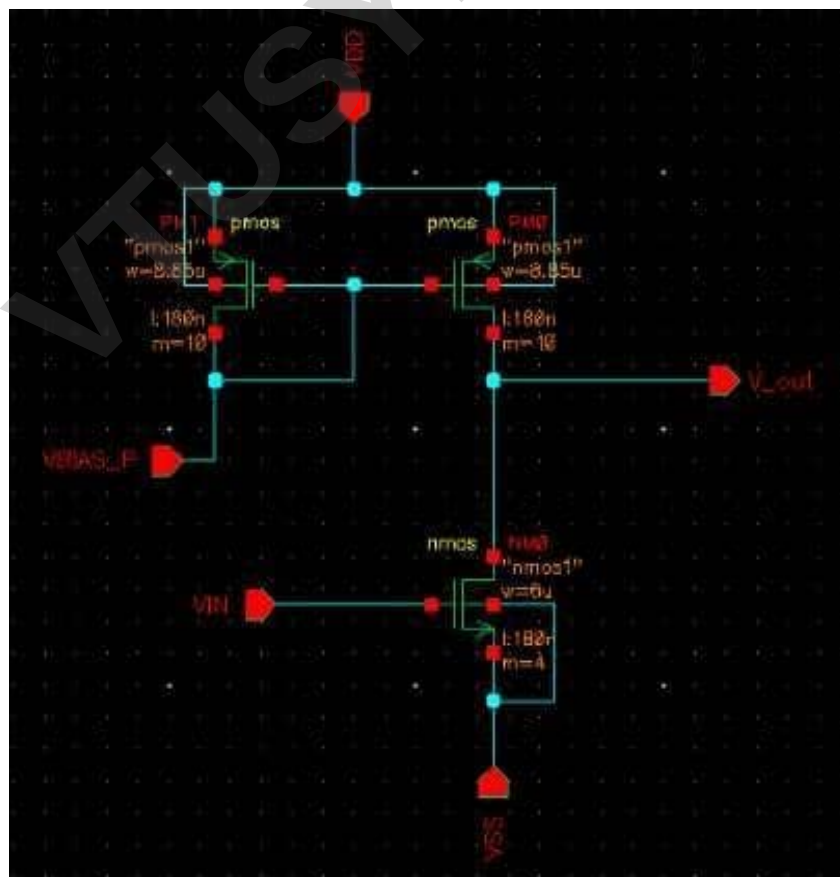
Experiment 9: Common Source Amplifier with PMOS Current Mirror Load.

Aim: a) To Construct the schematic of Common Source Amplifier with PMOS Current Mirror Load and find its transient response and AC response, by measuring the Unit Gain Bandwidth (UGB), amplification factor by varying transistor geometries, study of the impact of variation in width to UGB. b) Creating the Layout of common source amplifier, using optimum layout methods. And verification for DRC & LVS, extraction of parasitic and performing post layout.

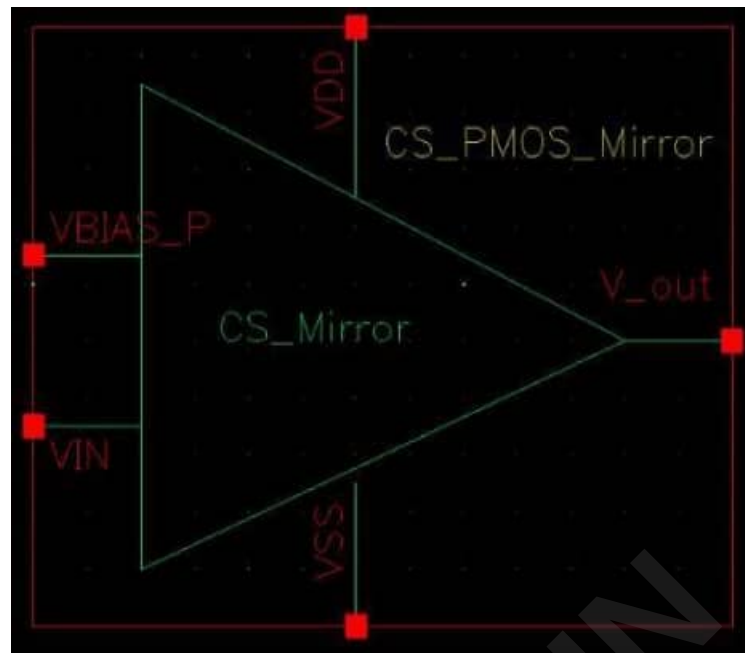
Theory:

A common source amplifier with a PMOS current mirror load is a fundamental analog circuit that provides high voltage gain. In this configuration, an NMOS transistor functions as the main amplifying device, with its gate receiving the input signal and its source connected to ground. The drain of the NMOS is connected to the drain of a PMOS transistor, which forms the output node. Two PMOS transistors are used in a current mirror configuration, where one is diode-connected and biased using an external voltage to set a reference current. This reference is mirrored to the other PMOS transistor, which acts as an active load for the NMOS. As the input voltage increases, the NMOS conducts more current, causing a voltage drop at the output node due to the constant current from the PMOS load. This results in an amplified and inverted output signal. The PMOS current mirror provides a high output resistance, enhancing the overall gain of the amplifier and making it efficient for analog signal processing in CMOS circuits.

Schematic of Common Source Amplifier with PMOS Current Mirror Load:

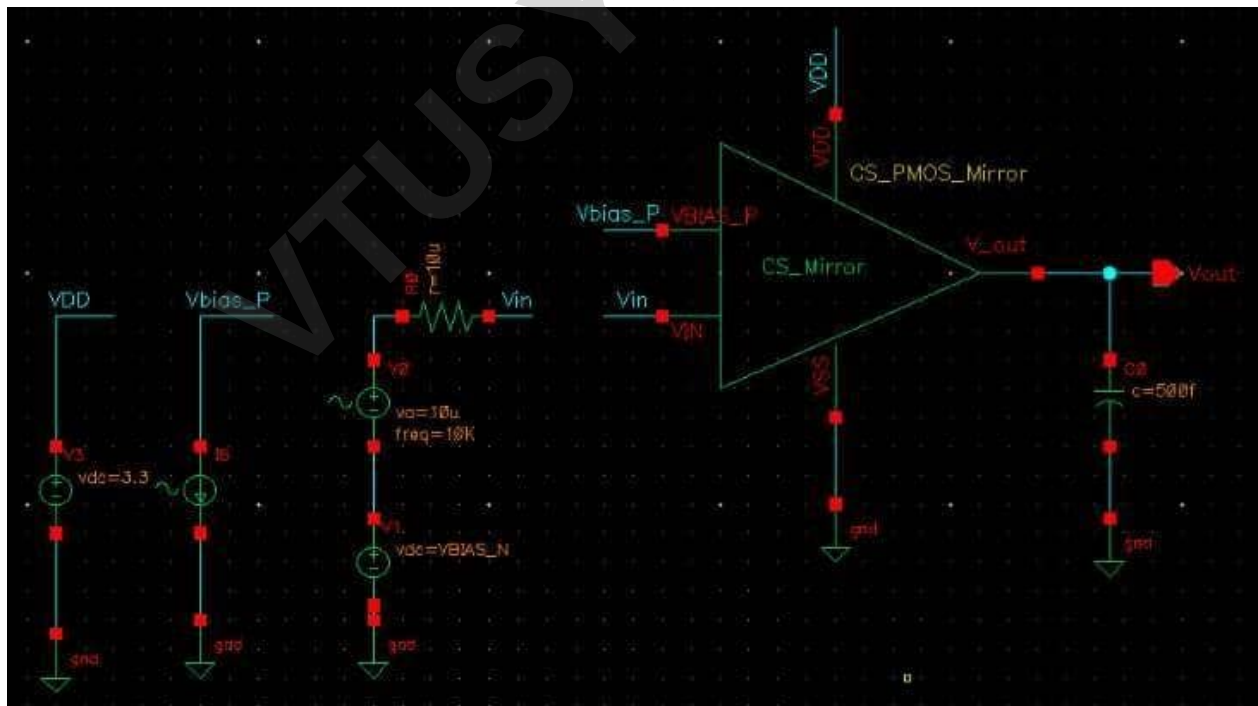


Symbol of Common Source Amplifier with PMOS Current Mirror Load:

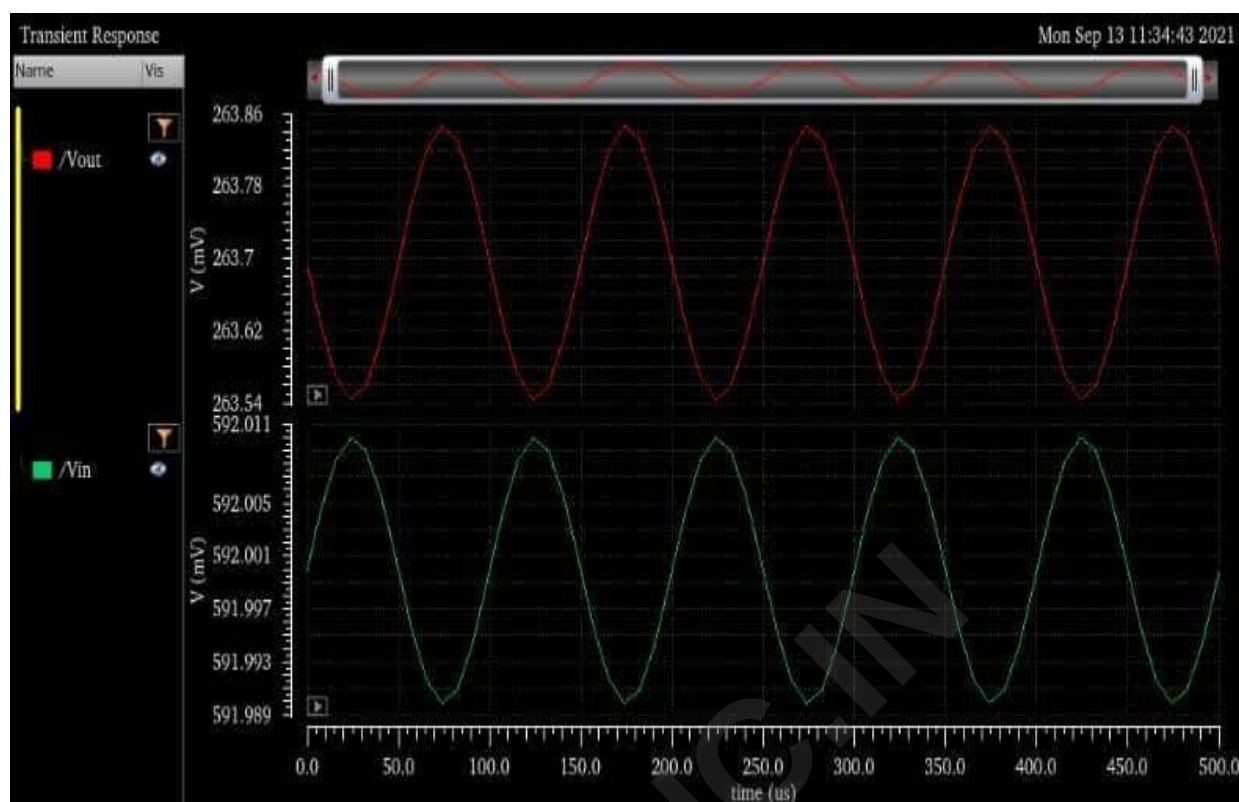


FUNCTIONAL SIMULATION:

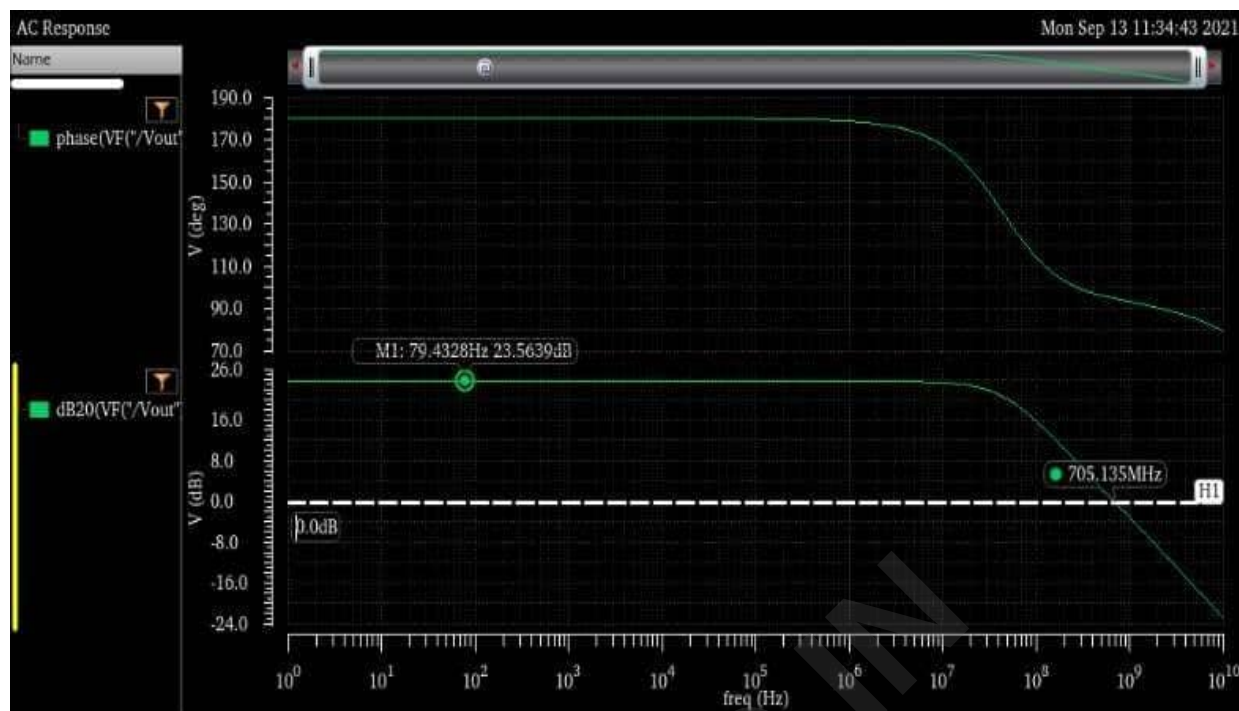
Using the symbol created, build the Test Schematic. Create a new cell view, instantiate the symbol of the common source amplifier with the PMOS current mirror load, DC voltage source, current source, AC voltage source, capacitance, resistance, and ground, and connect the wires.



Waveform of Transient and AC Analysis:



Gain and Phase Plot:



Layout of Common Source Amplifier with PMOS Current Mirror Load

